

Faculty of Engineering and Technology
Computer Science Department
Comp438 – QA course
Course Main Project

Malak Mustafa – 1212986

Rawa Hajaj - 1191497

Layan Elayyan - 1201593

Nahed Najjar 1220704

Dr. Mohammad HussainAli

Section 1

Date: 16/6/2025

Table of Contents

Introduction 4

Requirements Analysis 4

Functional Requirements 4

Non-Functional Requirements 5

Test Plan 6

Introduction 6

Scope 7

Test Objectives 7

Test Strategy 7

Test Schedule 8

Test Resources 8

Test Deliverables 8

Risks and Assumptions 8

Dependencies 8

Exit Criteria 8

Approvals 8

Appendices 9

Static Testing 9

Black box testing: 32

Test Cases and Test Execution 35

TC01: Adding an emergency contact (**link in Jira**) 35

TC02: Deleting an emergency contact(**link in Jira**) 36

TC03: Validate invalid work email editing(**link in Jira**) 37

TC04: Edit password with only numbers (**link in Jira**) 38

TC09: Add Document Type with Valid Input (**link in Jira**) 39

TC10: Search for Existing Document Types**(link in Jira)** 40

TC11: Edit an Existing Document Type **(link in Jira)** 41

TC12: Validation: Add Document Type with Empty Name**(link in Jira)** 42

TC-5: Negative value entered for project work hours (link in Jira) 43
TC-6: Calculate Working hours (**link in Jira**) 44
TC-18: Verify access as supervisor (**link in Jira**) 45
TC-19: Create previous timesheet (**link in Jira**) 46
TC-13: Invite an employee (**link in Jira**) 47
TC-14: Add new employee (**link in Jira**) 48
TC-15: Edit Employee Details (**link in Jira**) 49
TC-20: Delete a Language from Employee Profile (**link in Jira**) 50
Summary of the Test Cases 51
White Box Testing 52
Test Automation 63
Performance & Load Testing 74
Challenges & Solution 77
Conclusion 77

Introduction

This report aims to document the quality testing and performance improvement process for IceHrm, an open-source Human Resource Management (HRM) system used to organize employee-related administrative processes, including attendance management, leave, recruitment, and payroll. This project aims to ensure the quality and performance of the site through systematic testing that includes requirements analysis, test case development, automation, and performance testing.

With this work, we aim to ensure the IceHrm application's efficiency and stability, as well as its ability to smoothly and securely meet user needs and administrative workflow requirements.

Requirements Analysis

The IceHrm is free online HR Software with Leave Management, Recruitment, Payroll, and all other HR needs, such as timesheet, attendance, and documents. Analyzing both functional and nonfunctional requirements is essential to ensure good operation and comprehensive testing.

Functional Requirements

- **R1: Daily Attendance Recording (Punch In / Punch Out)**

  - The user should be able to record daily attendance and departure by pressing the "Enter" and "Out" buttons, automatically calculating the number of hours worked.

- **R2: View Attendance Reports**

  - The user should be able to view reports of entry and exit times and the number of hours worked during a specific period.

- **R3: Employee Profile Management**

  - User should be view, delete, and update profile information such as name, email, phone number, job title, and salary.

- **R4: Task Management**

  - The user should be able to create and assign tasks to employees, specify deadlines, and track their implementation.

● **R5: Edit profile**

○ The user should be able to modify their personal information, such as their name, date of birth, and insert a personal photo etc.

● **R6: Manage Document:**

○ The user should be able to add, edit, or copy a document.

● **R7: Search‹Filter**

○ The User should be able to search and filter capabilities to enable the user to easily find the required records within the various lists.

● **R8: Dashboard**

○ The user can see a quick overview of information by task and user and view alerts.

● **R9: Download Reports:**

○ The user can download reports on employee data, vacations, attendance, salaries, etc.

Non-Functional Requirements

Usability:

- R1: The system shall provide a user-friendly and easy-to-use interface.
- R2: The system shall support Role-based dashboards and easy navigation to access various modules (employee management, payroll, attendance, etc.)

Security:

- R3: The system provides a secure platform for storing and organizing employee information while allowing controlled access and data management.

Performance:

- R3: The system provides instant access to employee records with fast retrieval and search.
- R4: The system shall ensure consistent response times and stable performance regardless of the total number of active users.
- R5: The system shall allow employees and admins to upload and retrieve documents quickly.

- R6: The system shall support filtering documents based on document type or employee name for fast searchability.

Scalability:

- R7: The system shall be able to handle organizations of any size, from small businesses to large ones (50,000 users) without any decrease in performance.
- R8: The system shall allow the addition of new employees, modules, and business locations without affecting the performance.

Extensibility:

- R9: The system shall support the integration with other modules (Document attendance, asset, and expense management).
- R10: The system shall support extending using custom modules.

Maintainability:

- R11: The system shall allow feature custom solutions or extensions to be developed based on client-specific requirements.

Compatibility:

- R12: The system should support various devices, operating systems, and browsers.

Reliability:

- R13: The system shall provide a data backup and recovery mechanism to prevent data loss.

Test Plan

This section contains a structured test plan for the IceHrm website.

Introduction

This test plan outlines the general testing methodology, testing scope, objectives, activities, and resources required for a successful validation of the IceHrm system. The goal is to test the system's performance and functionalities to make sure it works reliably, securely, effectively, delivers the intended business value to its stakeholders, and meets both functional and nonfunctional requirements.

Scope

All of the testing tasks required to verify and validate the system are covered in this test plan. The system's usability, performance, and functionality will all be examined. Every module, including those for personnel management, time management, travel management, and personal information, is also tested. This does not include testing any third-party integrations or components that affect the functionality of the system.

Test Objectives

The main goal of this testing is to ensure that the system satisfies the technical and business requirements, operates as expected, and offers a dependable and secure user experience. The following specific objectives have been identified:

- Functional testing objectives:

This verifies that all functionalities of the system work as specified in the requirements, including the ability to log in, create, edit, and delete employee records in the Employee Management module, submit and approve leave requests in the Travel Management module, and update personal information. Additionally, ensure that each module handles both invalid and valid data correctly.

- Performance testing objectives:

This ensures that the system maintains acceptable response times under both normal and peak load conditions. Additionally. It verifies that the system can handle many concurrent users interacting with the system.

- Security Testing objectives:

This verifies that only authorized users have access to specific modules and data, with proper implementation of authentication, authorization, and access control.

- Usability Testing objectives:

This verifies that the system provides a user-friendly and intuitive interface, ensuring simple and consistent navigation across modules.

Test Strategy

- **Approach:** The agile testing approach will be used with manual and automated testing.
- **Testing levels:** Validation of the overall IceHrm system.

- **Techniques:** The black-box testing for user scenario by Katalon, load testing using Jmeter for concurrent users.

Test Schedule

- **Functional testing:** starts on [01/06/2025] and ends on [04/06/2025].
- **Performance testing:** starts on [08/06/2025] and ends on [11/06/2025].
- **Usability testing:** starts on [12/06/2025] and ends on [14/06/2025].

Test Resources

- **Hardware:** Testing will be conducted on various hardware configurations (laptops) and browsers (Chrome) to ensure cross-platform consistency.
- **Software:**
- **Katalon:** For automating document creation and validation flows.
- **JMeter:** For performance testing, simulating multiple admin users accessing or uploading documents.
- **Jira + AIO Plugin:** For managing test cases, executions, and linking bugs.

Test Deliverables

- Document structure: Test plan, test cases (black and white box), bug reports, test automation, performance testing.

Risks and Assumptions

- Assumptions: The test environment will be fully set up and stable before testing begins.
- Risks: The test environment may not match the production environment exactly, leading to inaccurate results.

Dependencies

- External dependencies: The testing schedule is dependent on the completion of development tasks.

Exit Criteria

- Exit criteria testing will be considered complete when: No critical issues are found in the final test, all functional, performance, security, and usability tests have been executed.

Approvals

- The test plan requires approval from: project manager and the QA manager.

Appendices

- Glossary: The test case is a specific set of inputs, execution conditions, and expected results developed to verify a particular feature.
- Reference: https://icehrm-open.gamonoid.com/login.php?next=P2c9bW9kdWxlcyZuPXJlcG9ydHMmbT1tb2R1bGVfVXNlcl9SZXBvcnRz

Static Testing

Checklist to check against the code

Checkpoint	**Explanation**
Clarity	Are class names and methods self-explanatory?
Commenting	Are there sufficient comments?
Duplication	Any repeated code or operations?
Error handling	Are errors handled gracefully?
Unused variables or operations	Are there any unused declared variables?
Maintainability	Is the code modular and easy to update?

Layan

Class Name: ApprovalStatus.php

Class location: core/src/Classes/Approval/ApprovalStatus.php

Purpose of Class: Manages approval status constants and validation logic.

Class Overview:

The “ApprovalStatus.php” class is responsible for managing and controlling approval chains within the IceHrm system. It handles the creation, retrieval, and status updates of employee-related approvals such as leave requests, based on multiple hierarchical levels of approvals.

Purpose of static testing:

Code inspection without running the program to ensure the program is written correctly, to find as many errors as possible to make sure the program is clear of bugs, to enhance the program, to make it easy to understand, not complex, and to check the quality of writing.

1. Class name space definition and class importation

```php
namespace Classes\Approval;

use Classes\BaseService;
use Classes\IceResponse;
use Employees\Common\Model\Employee;
use Employees\Common\Model\EmployeeApproval;
use Model\BaseModel;
use Utils\LogManager;
```

In this section, the name space of the class is defined, and the name space is used properly. Then, the importation of other classes to use in this code. All the classes have been imported properly, and all of them have been actively used in the class.

Table summarization of each class and its role:

Class Name	Description	Used in the class
BaseService	Helps retrieve the class name of the model	✅ Yes
IceRespose	Used to send a standard response (success or error)	✅ Yes
Employee	Represents employee data. Including approver fields	✅ Yes
EmployeeApproval	Represents approval chain records (status, approver level)	✅ Yes
BaseModel	A base class for all data models in the system.	✅ Yes
LoadManager	Log errors/warnings (ex., failed approval chain initialization)	✅ Yes

2. Class, and constants definitions

```
class ApprovalStatus
{

    const APP_ST_APPROVED = 1;
    const APP_ST_REJECTED = 0;
```

In this section, the class ApprovalStatus is declared. The class name is meaningful and indicates that it handles the logic related to the approval status process. In the class, two constants have been defined: Approved and Rejected. The constant names are clear and descriptive.

3. Singleton pattern

In this part of the class, a static variable and some functions have been defined for the singleton pattern, where only one instance of the class will exist.

```
  private static $me = null;

  private function __construct()
  {
  }

  public static function getInstance()
  {
      if (empty(self::$me)) {
          self::$me = new ApprovalStatus();
      }

      return self::$me;
  }
```

The static variable $me is initialized to hold the single instance of the class. the variable declaration is fine, but the name of the variable could be confusing or unclear for new developers. It can be named another name, for example, ($ApprovalStatusInstance).

The declaration of the construct and the get Instance functions are clear with no issues. The get Instance function, as the name indicates it creates a new instance of the class, so the name of the function is descriptive. The implementation of the singleton pattern and the constant declarations was properly used and correct.

4. Main Functions of the class

1. Is the Direct Approval function:

Purpose: The function's goal is to check if the employee has no approvers (direct approval).

```php
public function isDirectApproval($employeeId)
{
    $emp = new Employee();
    $emp->Load("id = ?", array($employeeId));
    if (empty($emp->approver1) && empty($emp->approver2) && empty($emp->approver3)) {
        return true;
    }
    return false;
}
```

Code review:

The definition of the function is correct and clear. The name of the function could be more descriptive than its actual name, for example, it can be HasNoapproval for better readability and clarity.

The function creates a new instance in $emp variable, then it loads the employee data from the database based on the employee's ID.

Then it checks if the approver1, approver2, and approver3 are empty, which indicates if the employee has no approvers.

The implementation of the function is clear and easy; it has no syntax or logic errors. however, the function lacks validation for cases where the employee ID does not exist in the database or fails to load the data.

2. Get Resolved Status:

Purpose: The function retrieves all resolved approvals records (approved or rejected) for a specific item and ID. The results are sorted by the order level.

```php
public function getResolvedStatuses($type, $id)
{
    $employeeApproval = new EmployeeApproval();
    $eas = $employeeApproval->Find("type = ? and element = ? and status > -1 order by level", array($type, $id));
    return $eas;
}
```

Code review:

The definition of the function is clear, the name of the function is clear and descriptive, it describes what the function does.

The implementation is clear; it starts with the creation of a new instance of EmployeeApproval() class. Then it finds all resolved approvals. Stores them in an array $eas; and returns the array of matching approvals sorted by the level.

The implementation is correct and simple; however, the code is missing the input validation of ID and type. It also lacks error handling when the Find() function fails or returns null. And it might return the array empty or null, or false.

3. Approval Chain Exists:

Purpose: The function's goal is to check if there are approval chain exists for the type and ID number of the request.

```php
public function approvalChainExists($type, $id)
{
    $list = $this->getAllStatuses($type, $id);
    return count($list) > 0;
}
```

Code review:

The function definition is correct, the name of the function is descriptive and clear, and it tells clearly what the function does.

It calls the function get All Statuses to retrieve all approval status records related to a specific element (by type and ID). And store them in a list array.

Then it checks if any approval records exist for that item by checking the length of the array. However, the code is also missing the input validation of the type and ID, and error handling if the list is empty, or if get all statuses returns false or a null value, or an unexpected type. The function always returns an array, even if it's empty.

4. getAllStatuses()

Purpose: The function gets all statuses for a specific request by type and ID number.

```php
public function getAllStatuses($type, $id)
{
    $employeeApproval = new EmployeeApproval();
    $eas = $employeeApproval->Find("type = ? and element = ? order by level", array($type, $id));
    return $eas;
}
```

Code review:

The function definition is correct, and the name is descriptive. It creates a new instance of the EmployeeApproval class model, which manages approval data. And uses the Find() function to fetch all approval records that match the type and ID, the results are ordered by level. Then it returns the list of matching approval records.

However, the code is also missing the input validation, or error handling if the Find function fails or returns an error. And if the eas array returns empty or null. Also, the eas array name is not descriptive, and it could be renamed for better readability.

5. Initialize Approval chain:

Purpose: As the name indicates, the function initializes the approval chain for a specific request by type and ID.

```php
public function initializeApprovalChain($type, $id)
{
    /* @var \Model\BaseModel $element */
    $nsClass = BaseService::getInstance()->getFullQualifiedModelClassName($type);
    $element = new $nsClass();
    $element->Load("id = ?", array($id));
    $employeeId = $element->employee;

    for ($i = 1; $i < 4; $i++) {
        $approver = $this->getApproverByLevel($i, $employeeId);
        if (!empty($approver)) {
            $employeeApproval = new EmployeeApproval();
            $employeeApproval->type = $type;
            $employeeApproval->element = $id;
            $employeeApproval->approver = $approver;
            $employeeApproval->level = $i;
            $employeeApproval->status = -1;
            $employeeApproval->active = 0;
            $employeeApproval->created = date("Y-m-d H:i:s");
            $employeeApproval->updated = date("Y-m-d H:i:s");
            $ok = $employeeApproval->Save();
            if (!$ok) {
                LogManager::getInstance()->error("Error:".$employeeApproval->ErrorMsg());
            }
        } else {
            LogManager::getInstance()->error("Approver is empty level:".$i);
        }
    }
```

Code review:

The function retrieves the fully qualified model class name for the input type, then creates an object $element and loads the corresponding record using the given ID. It gets the employee ID from the loaded element and loops through 3 levels (from 1 to 3), fetching the corresponding approver for each level. It checks the approver record and sets the approver information (type, ID, approver, level, status as pending, and active = 0). It sets the created and updated dates and saves all the records in the database. Errors during saving or if the approver is missing are logged using LogManager.

The function has many issues to be solved, first, there is no input validation as the above methods, second, there is no return value, the function doesn't return any success/failure indication, Third, there is no error handling, if the $nsClass, $element or employee ID. is invalid

or doesn't exist. Fourth, the variable $employeeApproval could be renamed to $approval for simplicity and to avoid repeating the long class name in a variable. Fifth, Log messages can include more details to show exactly what is wrong and where. Finally, before creating new approval records, it should be checked if there is no approvals already existing for that element and level.

6. Update approval status:

Purpose: The function updates the approval status of an element for a specific employee; the name of the function is descriptive and clear.

Note: The function is too large, so it will be divided into parts: (load current and next approvals, check permissions, Update status of current approval, activate next approver if needed, refresh, final return

Part 1: Function definition and check if approval chain exists

In the first part, the function checks if there is an approval chain for this item. If there is no approval, it returns success with null values.

```
public function updateApprovalStatus($type, $id, $currentEmployee, $status)
{
    if (!$this->approvalChainExists($type, $id)) {
        return new IceResponse(IceResponse::SUCCESS, array(null, null));
    }
```

Code review:

The section here returns true if there is no approval chain, which might be confusing, as it could falsely imply that the approval process was completed or succeeded, instead of returning a generic success, it could be clearer to return a detailed message such as "No approval chain was found for this item" in order to improve readability.

Part 2: Validate status, load the element, and get all approval statuses

In this part of the function, the status input of the item is checked whether it's 0 (Rejected) or 1 (Accepted), then it dynamically loads the model class corresponding to the type and loads the record with the ID of the type. Then it gets all approval status and initializes the approval level to zero, then it loops to find any rejection, then it stops the loop, otherwise it finds the current active level.

```php
if ($status != 0 && $status != 1) {
    return new IceResponse(IceResponse::ERROR, "Invalid data");
}
/* @var BaseModel $element */
$nsClass = BaseService::getInstance()->getFullQualifiedModelClassName($type);
$element = new $nsClass();
$element->Load("id = ?", array($id));

$eas = $this->getAllStatuses($type, $id);
$level = 0;
//check if the element is already rejected
foreach ($eas as $ea) {
    if ($ea->status == 0) {
        return new IceResponse(IceResponse::ERROR, "This item is already rejected");
    } elseif ($ea->active == 1) {
        $level = intval($ea->level);
    }
}
```

Code review:

The code has some issues that can be improved. for example, in the first if statement, the error message is too generic; it can be more specific, like “status must be 0 (Rejected) or 1 (Accepted)” to give the user a clear message. The variable name $eas could be more descriptive if a more descriptive name were used. In the loop, two checks are done; the two checks can be separated instead of being in the same loop for more clarity and flexibility.

Part 3: Load current and next approvals (if they exist)

In this part, it loads the current approver in $currentAL, then the next approver is loaded in $nextAL if the current level is less than 3.

```php
$currentAL = null;
if ($level > 0) {
    $currentAL = new EmployeeApproval();
    $currentAL->Load("type = ? and element = ? and level = ?", array($type, $id, $level));
}

$nextAL = null;
if ($level < 3) {
    $nextAL = new EmployeeApproval();
    $nextAL->Load("type = ? and element = ? and level = ?", array($type, $id, intval($level)+1));

    if (empty($nextAL->id)) {
        $nextAL = null;
    }
}
```

Code review:

The code lacks error handling if Load() function fails, and If it fails the failure is silent, and error logged to trace of what went wrong, and it uses the magic numbers (0 and 3) which reduces the code readability and flexibility, it could be better to use a constant declared at the beginning of the class to determine the max level of approvals “MAX_APPROVAL_LEVEL = 3” and can be used in the checking statement. And there is no check if $currentAL is empty, like $nextAL.

Part 4: Check permission, and update of current approval

In this part, the function checks if the current employee is allowed to approve or reject the request by comparing their ID with the assigned approver’s ID. If the employee is authorized, then the code updates the approval status and saves it.

```php
//Check if the current employee is allowed to approve
if ($level > 0 && $currentEmployee != $currentAL->approver) {
    return new IceResponse(IceResponse::ERROR, "You are not allowed to approve or reject");
}

if (!empty($currentAL)) {
    //Now mark the approval status
    $currentAL->status = $status;
    $currentAL->Save();
}
```

Code review:

The error message is clear, however, in the second part, the $currentAL variable could be null or not loaded properly, it must be checked before attempting to update. Additionally, if an error

occurs during the update, there is no handling for it. Also, the code saves it with no confirmation that save() succeeded. Which may lead to silent failures.

Part 5: Activate the next approver if needed

This block activates the next approver $nextAL by assigning active to 1, and deactivates others by assigning active to 0. It checks if the next approver exists, and loops through all approval statuses (EmployeeApproval entries).

```php
if (!empty($nextAL)) {
    /* @var EmployeeApproval $ea */
    foreach ($eas as $ea) {
        if ($ea->id == $nextAL->id) {
            $nextAL->active = 1;
            $nextAL->Save();
        } else {
            $ea->active = 0;
            $ea->Save();
        }
    }
}
```

Code review:

The logic of the block is generally correct, but there are some issues. The save() operation is called inside the loop for every EmployeeApproval object, which is not efficient and can lead to performance issues, especially if the list is large, and no error handling is performed in case of failure.

Rawa

Static Testing Report – DocumentsModulesManager.php

File: modules/documents/user/api/DocumentsModulesManager.php

Original Code:

```php
<?php
namespace Documents\User\Api;

use Classes\AbstractModuleManager;

class DocumentsModulesManager extends AbstractModuleManager
{

    public function initializeUserClasses()
    {
        if (defined('MODULE_TYPE') && MODULE_TYPE != 'admin') {
            $this->addUserClass("EmployeeDocument");
            $this->addUserClass("PayslipDocument");
        }
    }

    public function initializeFieldMappings()
    {
    }

    public function initializeDatabaseErrorMappings()
    {
    }

    public function setupModuleClassDefinitions()
    {
    }
}
```

Static Code Review Findings:

Observation	**Explanation**	**Recommendation**
Clear use of namespace	Good for organizing the codebase	No change needed
Conditional check on MODULE_TYPE	Relies on a global constant whose source is unclear	Add documentation explaining possible values and origin

Missing DocBlocks	Functions lack descriptive comments	Add PHPDoc blocks to explain the purpose and behavior of each method
No error handling	addUserClass() calls do not handle failures	Add try-catch or error checks to handle unexpected behavior
Empty methods	initializeFieldMappings(), initializeDatabaseErrorMappings(), and setupModuleClassDefinitions() are empty	Add inline comments like // Not required or implement as needed
Method name could be clearer	initializeUserClasses() may be vague	Consider renaming to registerUserDocumentClasses() or add comment explaining

---

Suggested Code Revision with Improvements:

```
namespace Documents\User\Api;
use Classes\AbstractModuleManager;
/**
 * Module Manager for User-side Document Module.
 */
class DocumentsModulesManager extends AbstractModuleManager
{
    /**
     * Register document-related classes for non-admin users.
     */
    public function initializeUserClasses()
    {
        if (defined('MODULE_TYPE') && MODULE_TYPE != 'admin') {
            // Registering employee document classes
            $this->addUserClass("EmployeeDocument");
            $this->addUserClass("PayslipDocument");
        }
```

```
    }
    /**
     * Initialize field mappings for user document module.
     * Currently not required.
     */
    public function initializeFieldMappings()
    {
        // Not required for this module
    }

    /**
     * Map database error codes to user-friendly messages.
     * Currently not implemented.
     */
    public function initializeDatabaseErrorMappings()
    {
        // Not implemented yet
    }

    /**
     * Set up module class definitions for the document module.
     * Currently not used.
     */
    public function setupModuleClassDefinitions()
    {
        // Not implemented yet
    }
}
```

Summary:

- The class serves its purpose in registering user-specific document modules.

- Documentation is lacking, which may reduce readability and maintainability.

- Adding inline comments, DocBlocks, and basic exception handling would improve quality.

- Empty methods should be either implemented or marked clearly as intentionally unused.

Nahed

Class Name: EmployeeTimeSheet.php

Class location: icehrm/core/src/TimeSheets/Common/Model/EmployeeTimeSheet.php

Purpose of Class: Manages employee timesheet data and calculates total worked time.

Class Overview:

The `EmployeeTimeSheet.php` class is responsible for handling employee timesheet records in the IceHRM system.
It calculates total worked minutes, retrieves processed data including total time, and formats employee-related entries.
The class extends `BaseModel`, supporting object-oriented inheritance and code reuse.
Functions like `getTotalTimeMinutes()`

```
67      public function getTotalTime()
68      {
69          $totMinutes = $this->getTotalTimeMinutes();
70          $minutes = $totMinutes % 60;
71          $hours = ($totMinutes - $minutes) / 60;
72
73          return CalendarTools::addLeadingZero($hours)
74          . ":" . CalendarTools::addLeadingZero($minutes);
75      }
76
```

and `getProcessGetData()` are structured to support the business logic cleanly.

```php

    public function postProcessGetData($entry)
    {
        $entry->total_time = $this->getTotalTime();
        return $entry;
    }

```

Purpose of static testing:

To review and inspect the code without running it, ensuring that it is written correctly, readable, well-structured and free from bugs. It helps identify potential issues such as lack of validation, poor structure, or missing documentation.

Code Review:

1. getTotalTimeMinutes():
- Good logic, but lacks validation or exception handling.
- No input validation for `start_time` and `end_time`.

```php
    public function getTotalTimeMinutes()
    {

        $start = $this->date_start . " 00:00:00";
        $end = $this->date_end . " 23:59:59";

        $timeEntry = new EmployeeTimeEntry();
        $list = $timeEntry->Find(
            "employee = ? and ((date_start >= ? and date_start <= ?) or (date_end >= ? and date_end <= ?))",
            array($this->employee, $start, $end, $start, $end)
        );

        $seconds = 0;

        foreach ($list as $entry) {
            $secondsTemp = (strtotime($entry->date_end) - strtotime($entry->date_start));
            if ($secondsTemp < 0) {
                $secondsTemp = 0;
            }

            $seconds += $secondsTemp;
        }

        $totMinutes = round($seconds / 60);

        return $totMinutes;
    }
```

2. getProcessGetData($entry):
- Clearly named and performs its function correctly.
- Relies on existing values without checking validity.

```
76
77      public function postProcessGetData($entry)
78      {
79          $entry->total_time = $this->getTotalTime();
80          return $entry;
81      }
82
```

3. postProcessGetElement():
- Breaks the Single Responsibility Principle. Should be split.

```
83      public function postProcessGetElement($entry)
84      {
85          $entry->days = [];
86          $days = CalendarTools::getDaysBetweenDates($entry->date_start, $entry->date_end);
87          foreach ($days as $dayObj) {
88              $entry->days[] = [$dayObj->format('Y-m-d'), $dayObj->format("(D) d M")];
89          }
90          return $entry;
91      }
92
93      public function getModuleAccess()
94      {
95          return [
96              new ModuleAccess('time_sheets', 'user'),
97          ];
98      }
99  }
```

4. **Code lacks comments**:
- Functions are not documented with explanations.

5**. Use of date/time logic**:
- Better to use DateTime or DateInterval for safer handling.

Table Summary:

Checkpoint	Explanation	Implemented in the code
Clarity	Are class names and methods self-explanatory?	Mostly clear methods and variable names
Commenting	Are there sufficient comments?	No comments
Duplication	Any repeated code or operations?	No duplication found

Error handling	Are errors handled gracefully?	No error handling
Unused variables	Any unused declared variables?	None found
Maintainability	Is the code modular and easy to update?	Needs refactoring

Suggested Improvements:

1. Use `DateTime` and `DateInterval` instead of string operations.

2. Add input validation for all critical fields (`start_date`, `end_date`, `employee`).

3. Add inline and docblock comments to all functions.

4. Split functions like `postProcessGetElement()` to follow the Single Responsibility Principle.

Malak

Class Name: EmployeesActionManager.php

Class location: Employees\Admin\Api\EmployeesActionManager.php

Purpose of Class: Manages employee-related actions.

Project: IceHrm

Class Overview: This class contains part of the administrator's permission over the employee. It allows the activation, termination, deletion, and archiving of employee records in addition to supporting data extraction (such as downloading archived data).

Main Functions

1. terminateEmployee($req)
   Purpose: Terminates an employee by setting the status to "Terminated" and recording the termination timestamp.

```php
    public function terminateEmployee($req)
    {
        $employee = new Employee();

        $this->baseService->checkSecureAccess('delete', $employee, 'Employee', $_POST);

        $employee->Load("id = ?", array($req->id));

        if (empty($employee->id)) {
            return new IceResponse(IceResponse::ERROR, "Employee Not Found");
        }

        $employee->termination_date = date('Y-m-d H:i:s');
        $employee->status = 'Terminated';

        $ok = $employee->Save();
        if (!$ok) {
            return new IceResponse(IceResponse::ERROR, "Error occurred while terminating employee");
        }

        return new IceResponse(IceResponse::SUCCESS, $employee);
    }
```

Code review: The function is well-structured and includes proper validation to check for employee existence. It also correctly includes a secure access check (checkSecureAccess). However, error logging is missing, and the response handling is basic — additional context for failures could improve traceability. Overall, the function is concise and maintainable.

2. activateEmployee($req)
   Purpose: Reactivates a terminated employee by clearing the termination date and setting status to "Active".

```php
    public function activateEmployee($req)
    {
        $employee = new Employee();
        $employee->Load("id = ?", array($req->id));

        if (empty($employee->id)) {
            return new IceResponse(IceResponse::ERROR, "Employee Not Found");
        }

        $employee->termination_date = null;
        $employee->status = 'Active';
        $ok = $employee->Save();
        if (!$ok) {
            return new IceResponse(IceResponse::ERROR, "Error occurred while activating employee");
        }

        return new IceResponse(IceResponse::SUCCESS, $employee);
    }
```

Code review: The function is logically straightforward and easy to read. However, it lacks a checkSecureAccess() call, which is a security concern since any authenticated user might activate employees. Also, no logging is implemented for failure cases. Refactoring and consistency with the terminateEmployee() function would improve reliability.

3. deleteEmployee($req)
   Purpose: Archives all data related to an employee, saves it to an ArchivedEmployee object, and deletes the original employee record.

```php
    public function deleteEmployee($req)
    {

        $employee = new Employee();
        $employee->Load("id = ?", array($req->id));

        if (empty($employee->id)) {
            return new IceResponse(IceResponse::ERROR, "Employee Not Found");
        }

        $archived = new ArchivedEmployee();
        $archived->ref_id = $employee->id;
        $archived->employee_id = $employee->employee_id;
        $archived->first_name = $employee->first_name;
        $archived->last_name = $employee->last_name;
        $archived->gender = $employee->gender;
        $archived->ssn_num = $employee->ssn_num;
        $archived->nic_num = $employee->nic_num;
        $archived->other_id = $employee->other_id;
        $archived->work_email = $employee->work_email;
        $archived->joined_date = $employee->joined_date;
        $archived->confirmation_date = $employee->confirmation_date;
        $archived->supervisor = $employee->supervisor;
        $archived->department = $employee->department;
        $archived->termination_date = $employee->termination_date;
        $archived->notes = $employee->notes;

```

```php
    public function deleteEmployee($req)

        $mapping = '{"nationality":["Nationality","id","name"],'
            .'"employment_status":["EmploymentStatus","id","name"],"job_title":["JobTitle","id","name"],'
            .'"pay_grade":["PayGrade","id","name"],"country":["Country","code","name"],'
            .'"province":["Province","id","name"],"department":["CompanyStructure","id","title"],'
            .'"supervisor":["Employee","id","first_name+last_name"]}';

        $employeeEnriched = BaseService::getInstance()->getElement('Employee', $employee->id, $mapping, true);
        $employeeEnriched = BaseService::getInstance()->cleanUpAdoDB($employeeEnriched);

        $data = new \stdClass();
        $data->enrichedEmployee = $employeeEnriched;

        $data->timesheets = $this->getEmployeeData($employee->id, new EmployeeTimeSheet());
        $data->timesheetEntries = $this->getEmployeeData($employee->id, new EmployeeTimeEntry());

        $attendnace = $this->getEmployeeData($employee->id, new Attendance());
        $data->attendance = array_map(
            function ($item) {
                $item->image_in = '';
                $item->image_out = '';
            },
            $attendnace
        );

        if (class_exists('\Documents\Common\Model\EmployeeDocument')) {
            $data->documents = $this->getEmployeeData($employee->id, new \Documents\Common\Model\EmployeeDocument());
        }
```

```
76          public function deleteEmployee($req)
130             $data->travelRecords = $this->getEmployeeData($employee->id, new EmployeeTravelRecord());
131             $data->qualificationSkills = $this->getEmployeeData($employee->id, new EmployeeSkill());
132             $data->qualificationEducation = $this->getEmployeeData($employee->id, new EmployeeEducation());
133             $data->qualificationCertifications = $this->getEmployeeData($employee->id, new EmployeeCertification());
134             $data->qualificationLanguages = $this->getEmployeeData($employee->id, new EmployeeLanguage());
135             $data->salary = $this->getEmployeeData($employee->id, new EmployeeSalary());
136             $data->dependants = $this->getEmployeeData($employee->id, new EmployeeDependent());
137             $data->emergencyContacts = $this->getEmployeeData($employee->id, new EmergencyContact());
138             $data->projects = $this->getEmployeeData($employee->id, new EmployeeProject());
139             $data->expenses = $this->getEmployeeData($employee->id, new EmployeeExpense());
140
141             $archived->data = json_encode($data, JSON_PRETTY_PRINT);
142
143             $ok = $archived->Save();
144             if (!$ok) {
145                 LogManager::getInstance()->error('Error occurred while archiving employee :'.$archived->ErrorMsg());
146                 return new IceResponse(IceResponse::ERROR, "Error occurred while archiving employee");
147             }
148
149
150             $ok = $employee->Delete();
151             if (!$ok) {
152                 LogManager::getInstance()->error('Error occurred while deleting employee :'.$employee->ErrorMsg());
153                 return new IceResponse(IceResponse::ERROR, "Error occurred while deleting employee");
154             }
155
156             return new IceResponse(IceResponse::SUCCESS, $archived);
157         }
```

Code review: This is the most complex function in the class and involves extensive data gathering. It handles multiple modules and stores a full employee snapshot as JSON. The logic is comprehensive, but lacks an access control check and contains an issue where the attendance array is modified via array_map but not returned or saved. Additionally, temporary data structures and mappings could be abstracted to reduce cognitive load. Logging is used appropriately for critical failures.
Suggestion: This function can be two functions, archive and delete. Then call the archive function in the delete function, which can be less complex and more readable.

4. downloadArchivedEmployee($req)
   Purpose: Allows downloading an archived employee’s data in .txt format

```php
    public function downloadArchivedEmployee($req)
    {
        if ($this->baseService->currentUser->user_level != 'Admin') {
            echo "Error: Permission denied";
            exit();
        }

        $employee = new ArchivedEmployee();
        $employee->Load("id = ?", array($req->id));

        if (empty($employee->id)) {
            return new IceResponse(IceResponse::ERROR, "Employee Not Found");
        }

        $employee->data = json_decode($employee->data);
        $employee = $this->baseService->cleanUpAdoDB($employee);

        $str = json_encode($employee, JSON_PRETTY_PRINT);

        $filename = uniqid();
        $file = fopen("/tmp/".$filename, "w");
        fwrite($file, $str);
        fclose($file);

        $downloadFileName = "employee_".$employee->id."_"
            .str_replace(" ", "_", $employee->first_name)."_"
            .str_replace(" ", "_", $employee->last_name).".txt";

        header("Pragma: public"); // required
        header("Expires: 0");
        header("Cache-Control: must-revalidate, post-check=0, pre-check=0");
        header("Content-Description: File Transfer");
        header("Content-Type: image/jpg");
        header('Content-Disposition: attachment; filename="'.$downloadFileName.'"');
        header("Content-Transfer-Encoding: binary");
        header("Content-Length: ".filesize("/tmp/".$filename));
        readfile("/tmp/".$filename);
        exit();
```

Code review: The method performs its intended task but contains critical flaws. The content type header is set to image/jpg, which is incorrect for a text file. Also, the temporary file is not deleted after use, potentially causing disk clutter. The function is straightforward otherwise, but lacks proper error messaging and should use logging instead of raw echo.

5. getEmployeeData($id, $obj)

   Purpose: Private helper method that loads and cleans related employee data from various modules.

```php

    private function getEmployeeData($id, $obj)
    {
        $data = array();
        $objs = $obj->Find("employee = ?", array($id));
        foreach ($objs as $entry) {
            $data[] = BaseService::getInstance()->cleanUpAdoDB($entry);
        }
        return $data;
    }
}
```

Code review: This method is reusable, efficient, and ensures that the same code is not duplicated in the deleteEmployee() function. It uses a loop and the central cleaning

function cleanUpAdoDB. There are no major issues, but the method could be extended with logging for better traceability in case of empty or failed queries.

Summary

The code is functional and well-organized, but there is a lack of inline comments and method-level documentation, which may reduce readability for new developers. Additionally, certain security and maintainability aspects — such as missing permission checks in some methods, improper error logging, and repeated logic — affect the robustness of the code. Refactoring and adding more consistent error handling would improve maintainability significantly.

Table of summary

Checkpoint	**Implemented in the code**
Clarity	Class and method names are self-explanatory and follow naming conventions.
Commenting	No comments
Duplication	Employee loading is repeated across multiple functions.
Error Handling	Basic error messages are returned, but logging is missing in most methods.
Unused Variables	No unused variables were found
Maintainability	The code is modular, but can be improved.

Black box testing:

Layan

Personal information section

1. Equivalence portioning:

Valid/Invalid categories:

Email address:

Type	Example	Why
Valid	icehrm+manager@web-stalk.com	Valid email
Invalid	icehrm+managerweb-stalk.com	@ is missed

2. Boundary value analysis:

Test the edges of input ranges (minimum, maximum, below, above)

Password: The password should be 8 characters long, with at least one symbol, one uppercase letter, one lowercase letter, and one number.

1. Minimum 8 characters.

2. At least one uppercase letter

3. At least one lowercase letter.

4. At least one symbol.

5. At least one number.

Test cases	Password	Length	Contains an uppercase letter	Contains a lowercase letter	Contains a symbol letter	Contains at least one number	Valid/Invalid
TC01	1201583	7	No	No	No	Yes	Invalid Too short
TC02	120RTXXX	8	Yes	No	No	Yes	Invalid must include at least one lowercase
TC03	120RTXxX	8	Yes	Yes	No	Yes	Invalid must include at least one symbol
TC04	120RTXx,	8	Yes	Yes	Yes	Yes	Valid password
TC05	RTXx,vEs	8	Yes	Yes	Yes	No	Invalid must contain at least one number
TC06	RTXx,v2	7	Yes	Yes	Yes	Yes	Invalid Too short

3. Decision table:

Emergency Contacts: The user should enter at least the name to be added to the emergency contacts

2^5 = 32 possibilities

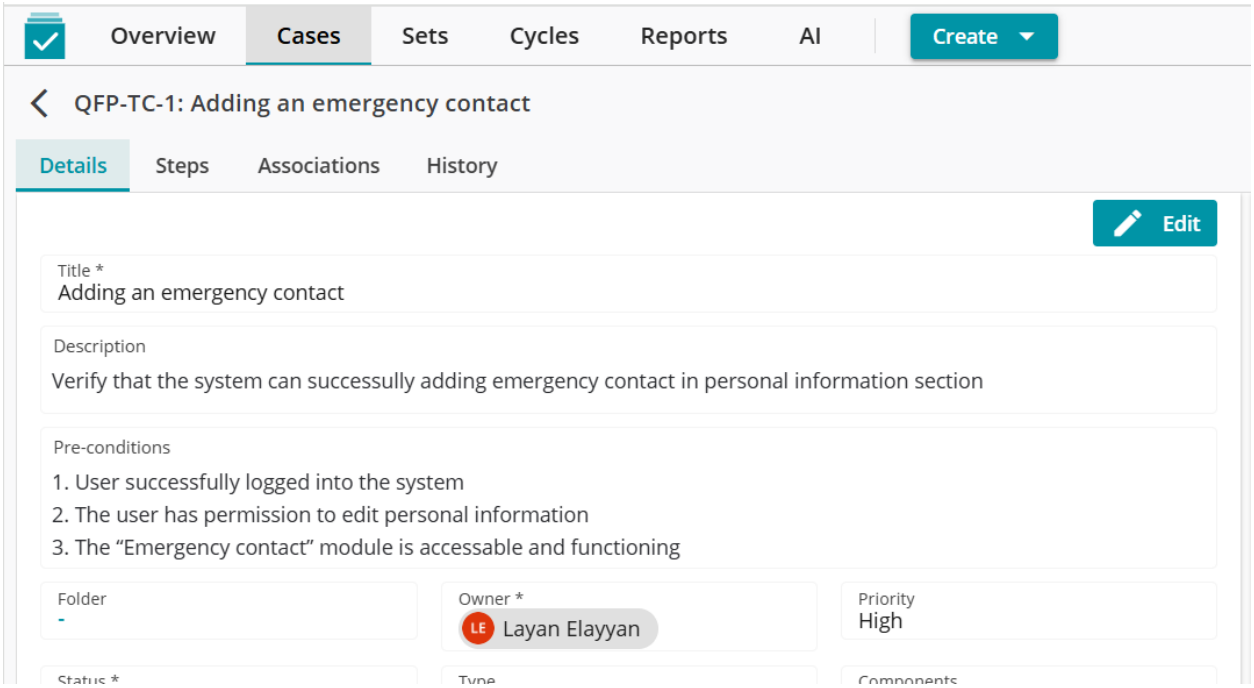
Condition	name	Relationship	Home phone	Work phone	Mobile phone	Added
TC01	Yes	Yes	Yes	Yes	Yes	Yes
TC02	No	Yes	Yes	Yes	Yes	No
TC03	Yes	No	Yes	Yes	Yes	Yes
TC04	Yes	Yes	No	Yes	Yes	Yes
TC05	Yes	Yes	Yes	No	Yes	Yes
TC06	Yes	Yes	Yes	Yes	No	Yes
TC07	No	No	Yes	Yes	Yes	No
TC08	No	No	No	Yes	Yes	No
TC09	No	No	No	No	No	No
TC010	Yes	No	No	Yes	Yes	Yes
TC011	Yes	No	No	No	Yes	Yes
TC012	Yes	No	No	No	No	Yes
TC013	Yes	Yes	No	No	Yes	Yes
TC014	Yes	Yes	No	No	No	Yes
TC015	Yes	Yes	Yes	No	No	Yes
TC016	No	No	No	No	Yes	No
TC017	No	No	No	Yes	No	No
TC018	No	No	Yes	No	No	No
TC019	No	No	Yes	No	Yes	No
TC020	No	No	Yes	Yes	No	No
TC021	No	Yes	No	No	No	No
TC022	No	Yes	No	No	Yes	No
TC023	No	Yes	No	Yes	No	No
TC024	No	Yes	No	Yes	Yes	No
TC025	No	Yes	Yes	No	No	No
TC026	No	Yes	Yes	No	Yes	No
TC027	No	Yes	Yes	Yes	No	No
TC028	Yes	No	Yes	No	No	Yes
TC029	Yes	No	Yes	No	Yes	Yes
TC030	Yes	No	Yes	Yes	No	Yes
TC031	Yes	Yes	No	Yes	No	Yes
TC032	Yes	Yes	Yes	Yes	No	Yes

Test Cases and Test Execution

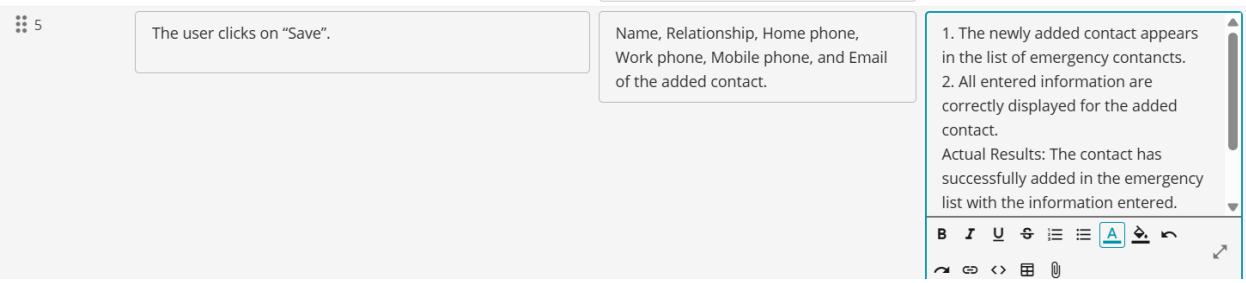
The section contains all the test cases; each group member created 4 test cases. The test cases were developed using black box testing to manage and document these test cases efficiently, and the team utilized Jira, a specialized testing and project management tool.

Layan

TC01: Adding an emergency contact (link in Jira)



1	The user navigates to “Personal information” section.		The peronal information menu expands, displaying its sub-sections
2	The user chose “Emergency Contacts” tap.		The "Emergency Contants" page opens
3	The user clicks on “Add New”.		The "Add New" contact form displayed
4	The user starts to fill the information required in the fields.	Name, Relationship, Home phone, Work phone, Mobile phone, and Email of the added contact.	The entered information appears in the field.



Analyze the Result:

Expected result: 1. The newly added contact appears in the list of emergency contacts.

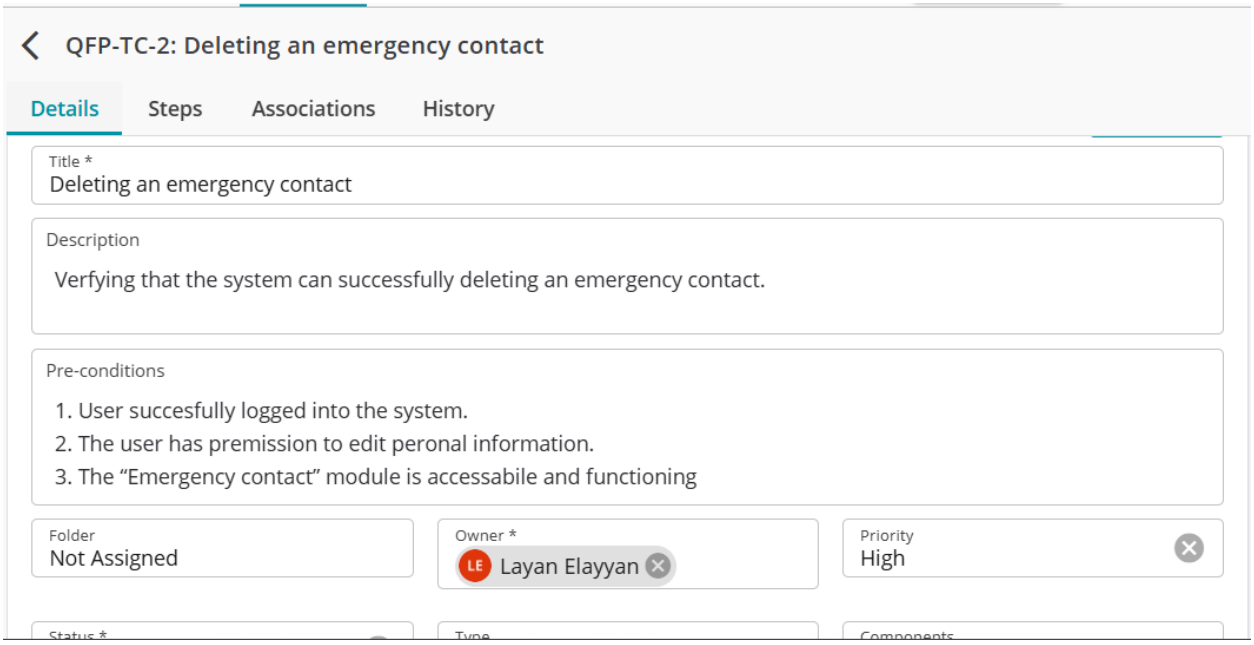
2. All entered information is correctly displayed for the added contact.

Actual result: The contact has been successfully added to the emergency list with the information entered.

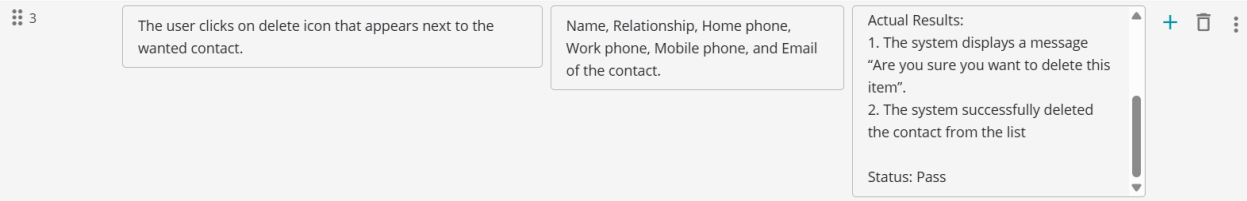
Detecting bugs report: No bugs detected.

Layan

TC02: Deleting an emergency contact(link in Jira)







Analyze the Result:

Expected result: 1. The system displays a message to ensure the deletion.

2. The system successfully deletes the contact from the emergency list with their information attached.

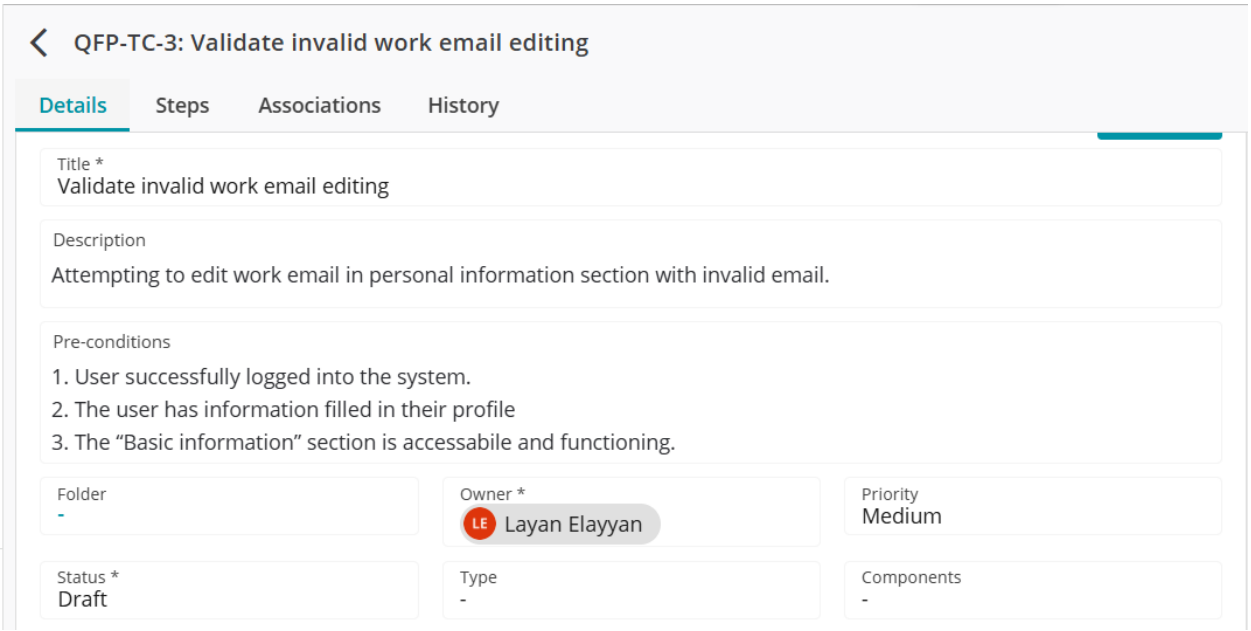
Actual result: 1. The system displays a message "Are you sure you want to delete this item?".

2. The system successfully deleted the contact from the list

Detecting bugs report: No bugs detected.

Layan

TC03: Validate invalid work email editing(link in Jira)



	Step	Data	Expected results
1	The user navigates to “Personal information” section.		The Personal Information menu expands upon clicking, displaying its sub-sections.
2	The user chose “Basic information” tap.		The "Basic information" page opens
3	The user clicks on “Edit” button in top of the page, or the “Edit” button next to Personal information” section.		The edit information form is displayed
4	The user clicks “Next” until the “Contact” section is displayed.		The form changes based on the section reached
5	The user tries to edit the work email into invalid eamil (for	User information (Work email)	The entered email appears in the field.
6	The user clicks on "Save" button	User information (Work email)	Expected Results: The system refues to save the edits made Actual Results: The system displays “Invalid value for Work email” and refues to save the changes made Status: Pass

Analyze the Result:

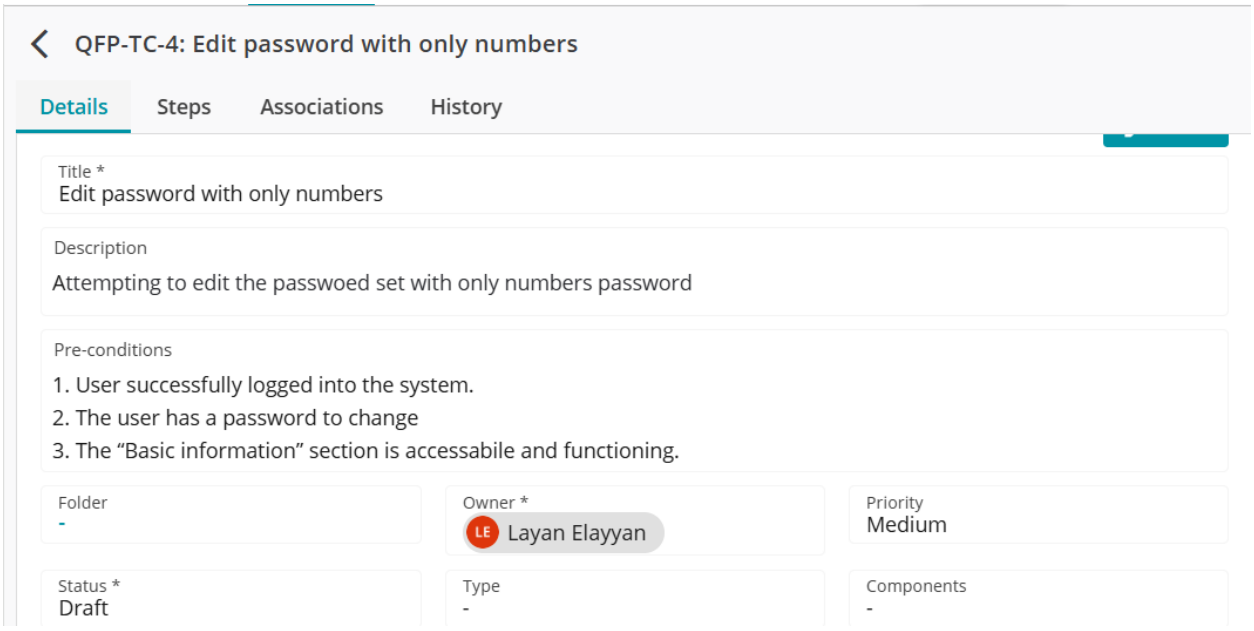
Expected result: The system refuses to save the edits made

Actual result: The system displays “Invalid value for Work email” and refuses to save the changes made

Detecting bugs report: No bugs detected.

Layan

TC04: Edit password with only numbers (link in Jira)



	Step	Data	Expected results
1	The user navigates to "Personal information" section.		The Personal Information menu expands upon clicking, displaying its sub-sections.
2	The user chose "Basic information" tap.		The Basic information page opens
3	The user clicks on "Update Password" button in top of the page.	User information (Password)	The update password form appears
4	The user types the current password in "Current Password" field.	User information (Password)	The entered password appears in the field (masked for security).
5	The user types the new password with only numbers.	User information (Password)	The system displays error message since the password cannot be only numbers
6	The user the confirm password.		Expected Results: The system refuses to save the edits made Actual Results: The system displays "Password must include at least one lowercase letter" and refuses to save the changes made

Analyze the Result:

Expected result: The system refuses to save the edits made

Actual result: The system displays "Invalid value for Work email" and refuses to save the changes made

Detecting bugs report: No bugs detected.

Rawa

TC09: Add Document Type with Valid Input (link in Jira)

1

Add Document Type with Valid Input

Description

Validate that the system successfully adds a new document type when all required fields are filled in with valid data.

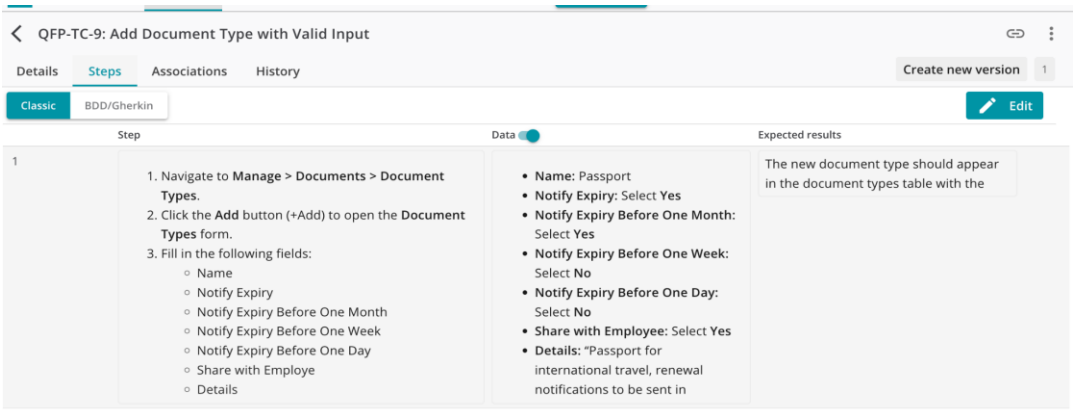
Pre-conditions

The user is logged in as Manager and on the Document Types page.

Key	QFP-TC-9
Owner	R Rawaa
Folder	-
Status	Draft
Priority	Medium
Type	Functional
Estimated Effort	-
Components	-
Releases	-
Tags	-
Version Created	Jun 14, 2025 02:08 PM
Case Created	Jun 14, 2025 02:08 PM
Updated	Jun 14, 2025 02:41 PM

Custom Fields

No custom fields defined



Analyze the Result:

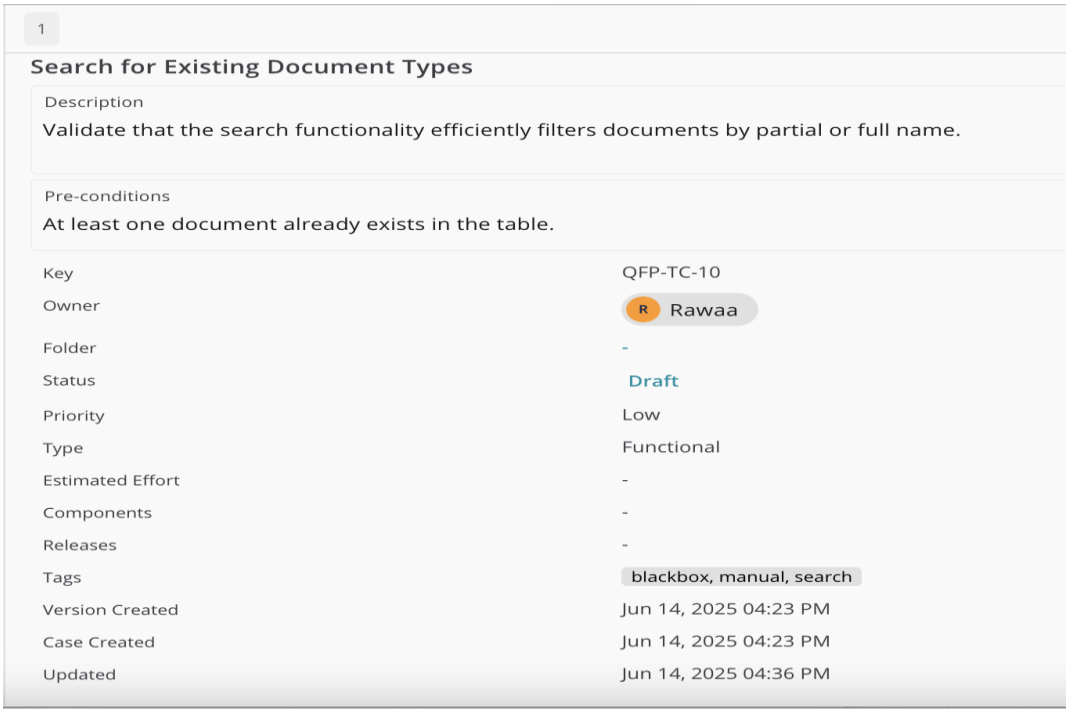
Expected result: A new document type with the name "Passport" and the entered details should appear in the document types table.

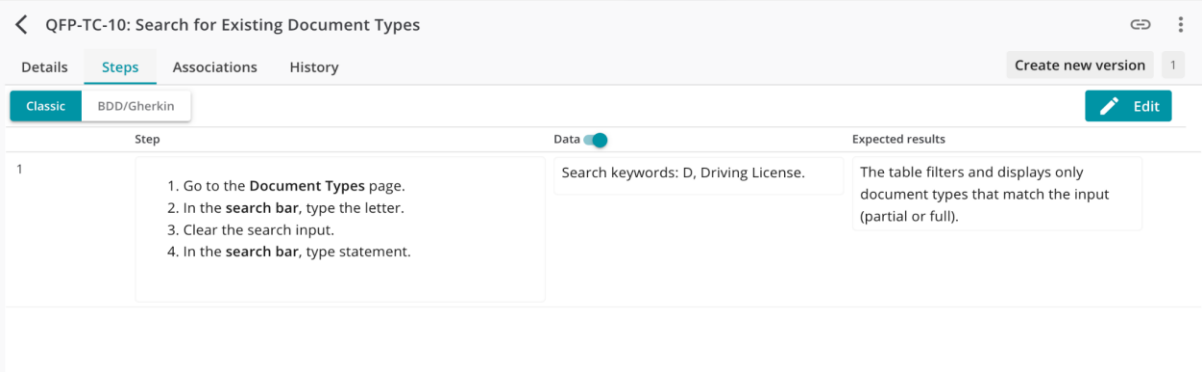
Actual result: The document type "Passport" was successfully added and appeared correctly in the table with all the selected options saved.

Detecting bugs report: No bugs detected.

Rawa

TC10: Search for Existing Document Types(link in Jira)





Analyze the Result:

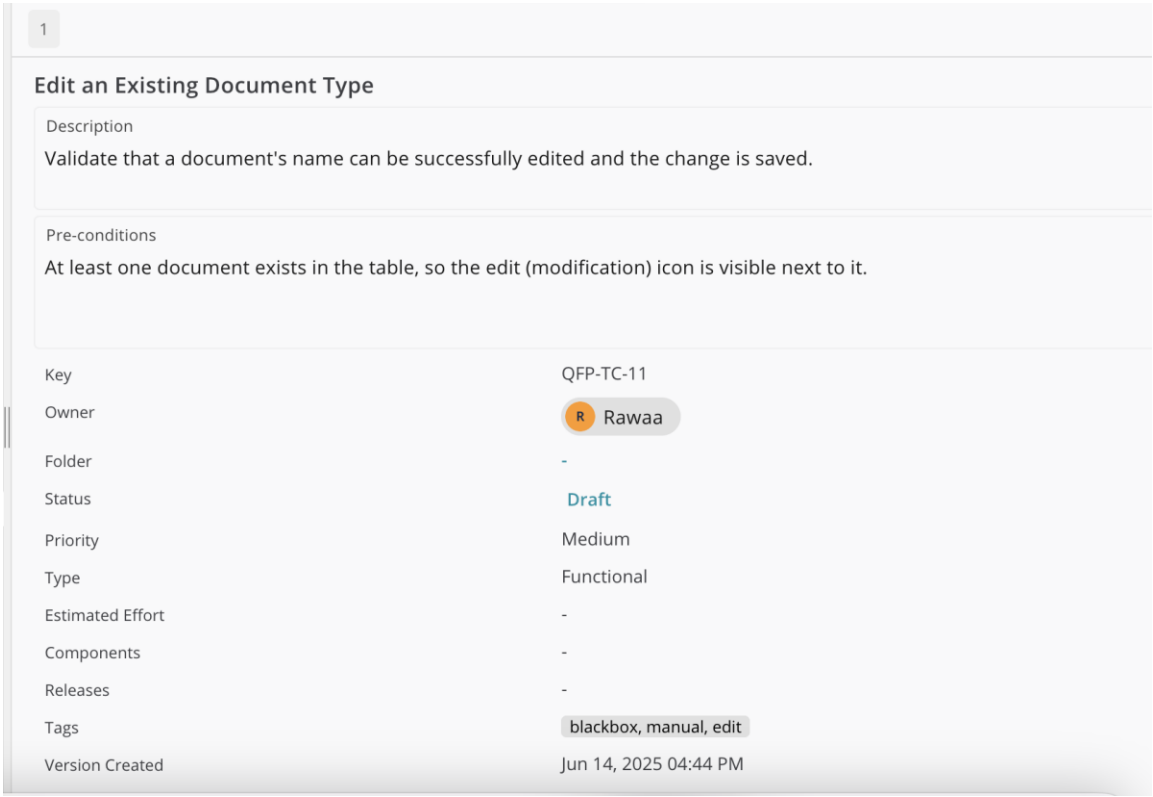
Expected result: The document types table should filter and display only entries that match the input keywords (e.g., "D", "Driving License").

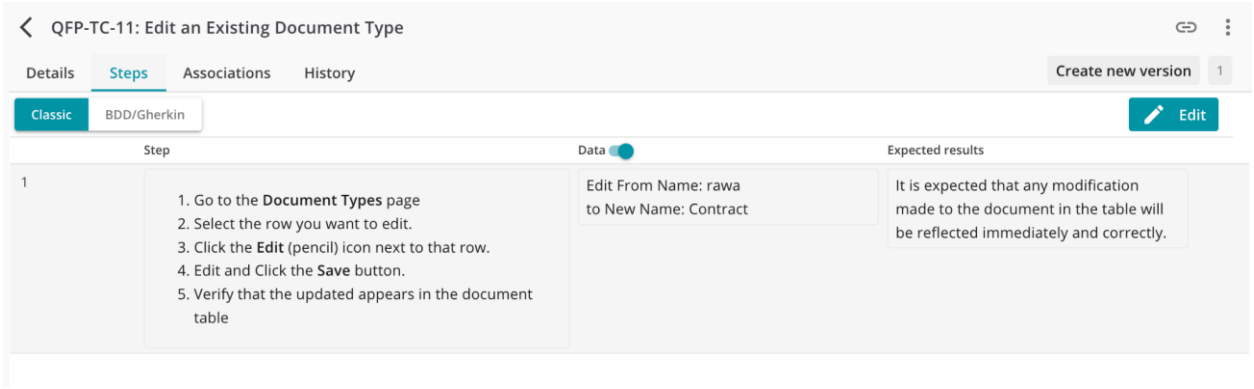
Actual result: The table successfully filtered the document list based on both partial ("D") and full ("Driving License") inputs.

Detecting bugs report: No bugs detected.

Rawa

TC11: Edit an Existing Document Type (link in Jira)





Analyze the Result:

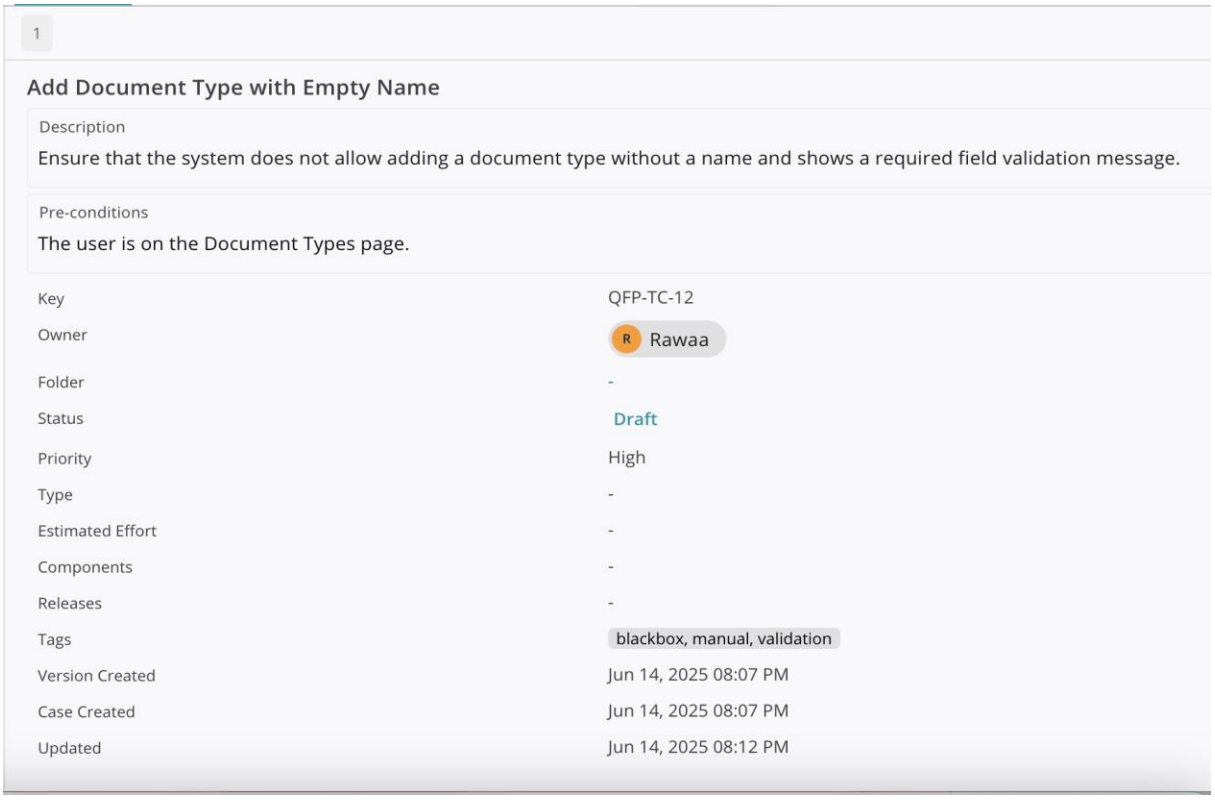
Expected result: The document name should be updated from "rawa" to "Contract", and the updated name should appear immediately in the table.

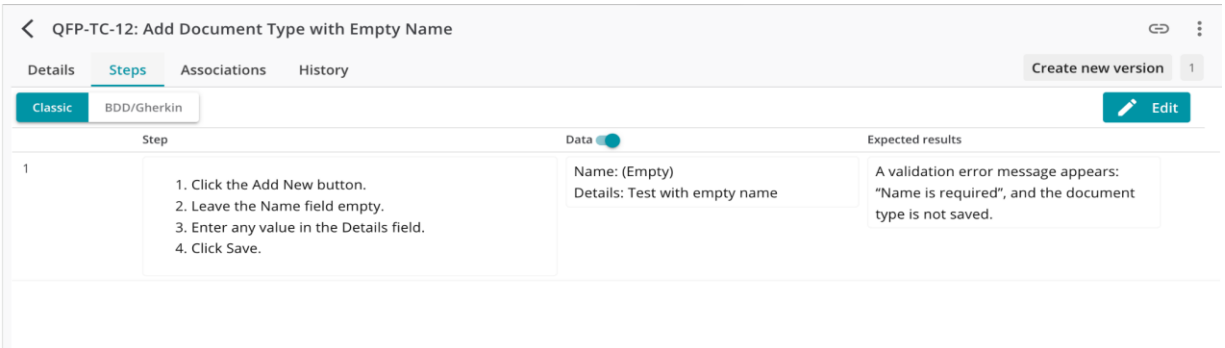
Actual result: The document was successfully edited. The name “Contract” appeared in place of “rawa” in the table after saving.

Detecting bugs report: No bugs detected.

Rawa

TC12: Validation: Add Document Type with Empty Name(link in Jira)





Analyze the Result:

Expected result: A validation error message "Name is required" should appear, and the document type should not be saved.

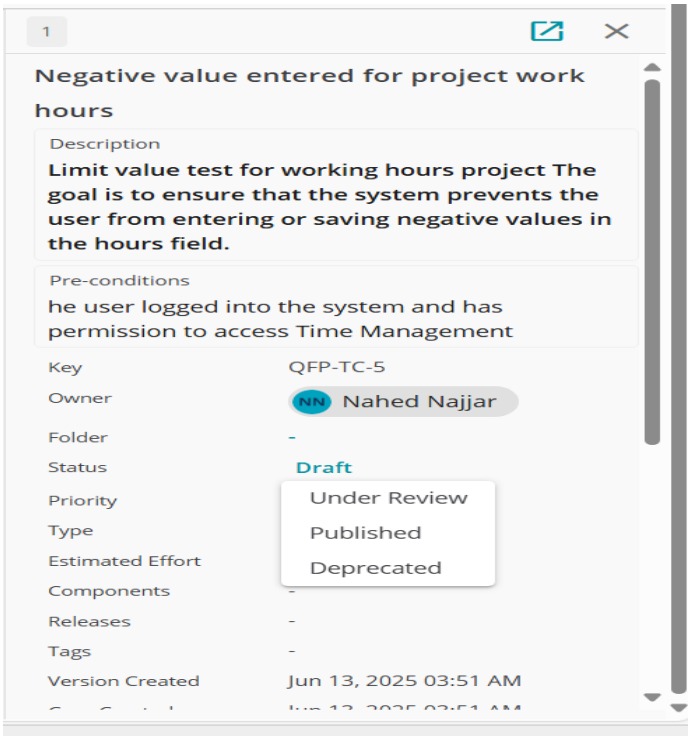
Actual result: The system displayed the message "Name is required" and prevented saving the document type.

Detecting bugs report: No bugs detected.

Nahed

Positive test case:

TC-5: Negative value entered for project work hours (link in Jira)



	Step	Data	Expected results
1	Log in as an manager		Go to Main Page
2	Go to Time Management		View Page properties
3	Go to Time Sheets		View Page properties
4	Edit Time Sheets Entry		Display projects and input fields
5	Enter a negative value	-5	Do not allow entry

Analyze the Result:

Expected result: It should regect the negative values

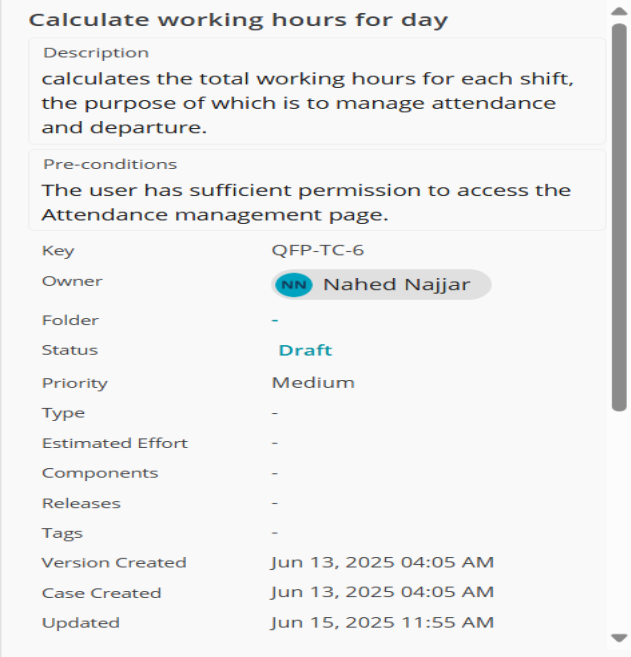
Actual result: negative value not displayed in field

Detecting bugs report: No bugs detected.

Nahed

Positive test case:

TC-6: Calculate Working hours (link in Jira)



	Step	Data	Expected results
1	Log in as an manager		Go to main page
2	Go To Time Management		view page properties
3	Go To Attendance Management		View attendance records
4	Click Punch in		Display input fields
5	Enter Time-in	2025 jun 9 8:30:00	insert time
6	Enter Time-out	2025 jun 9 16:24:00	insert time
7	Click View		Total Hours:7.9

Analyze the Result:

Expected result: It should display Total Hours:7.9 /8h

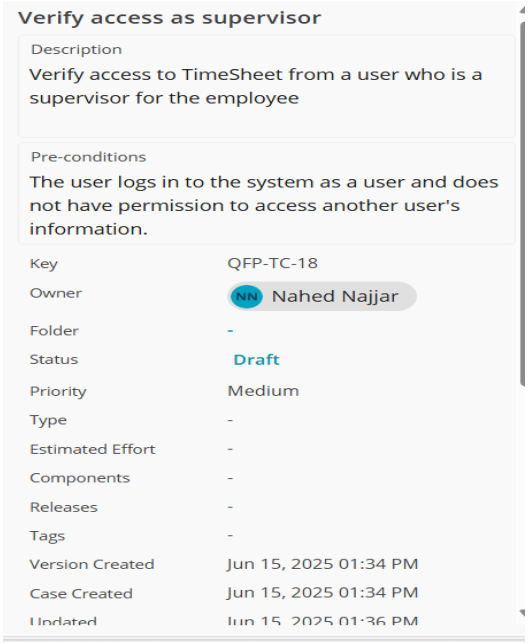
Actual result: Total Hours:7.9 /8h

Detect bugs report: No bugs detected**.**

Nahed

Positive test case:

TC-18: Verify access as supervisor (link in Jira)



	Step	Data	Expected results
1	1.user log into the system as a manager		view page properties
2	2.Go To Time Mangement		view page properties
3	3.View Timesheet		view TimeSheet for employee

Analyze the Result:

Expected result: It should display the Time Sheet

Actual result: display the Time Sheet

Detect bugs report: No bugs detected.

Nahed

Positive test case:

TC-19: Create previous timesheet (link in Jira)

Create previous timesheet

Description

Trying to create an existing timesheet

Pre-conditions

User has a permission to create a time sheet

Key	QFP-TC-19
Owner	NN Nahed Najjar
Folder	-
Status	Draft
Priority	Low
Type	-
Estimated Effort	-
Components	-
Releases	-
Tags	-
Version Created	Jun 15, 2025 02:11 PM
Case Created	Jun 15, 2025 02:11 PM
Updated	Jun 15, 2025 02:14 PM

	Step	Data	Expected results
1	user log in to the system as a user		view page properties
2	Go To Time Mangement		view page properties
3	Go To Time Mangement		view All My TimeSheets
4	Try to create previous TimeSheet		Error: Timesheet already exists

Analyze the Result:

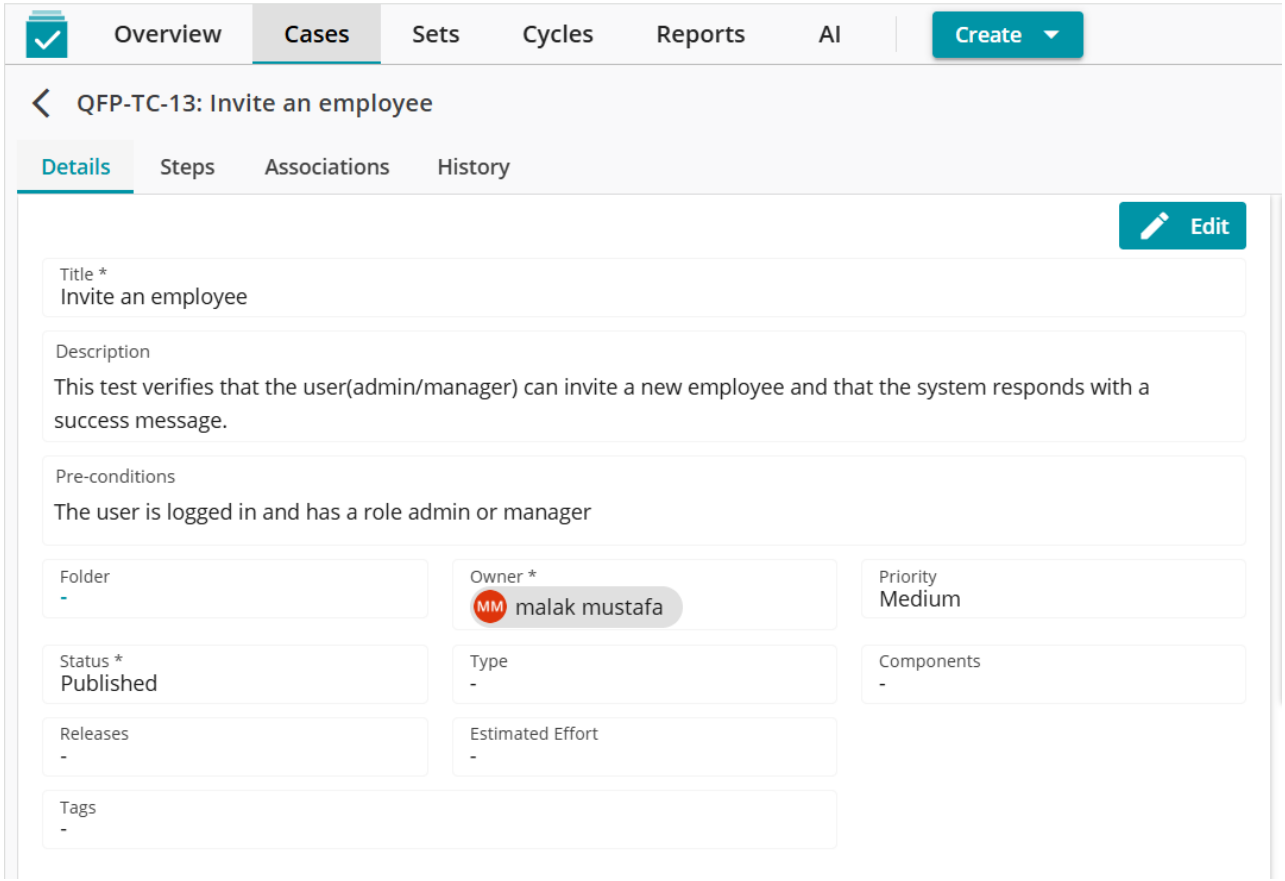
Expected result: Error: Timesheet already exists

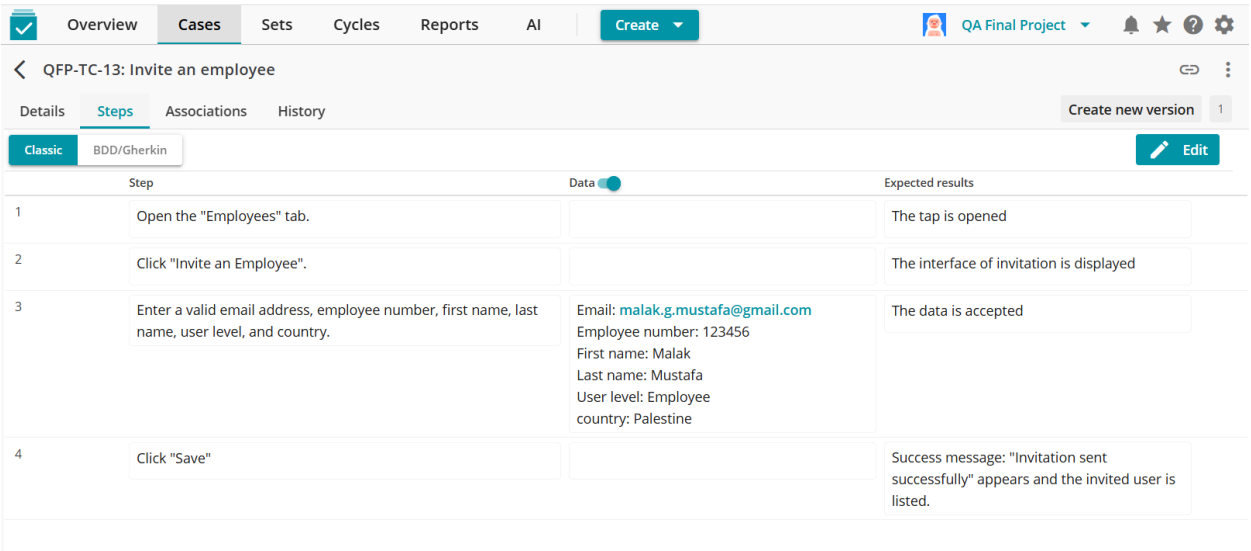
Actual result: Error: Timesheet already exists

Detect bugs report: No bugs detected.

Malak

TC-13: Invite an employee (link in Jira)





Analyze the Result:

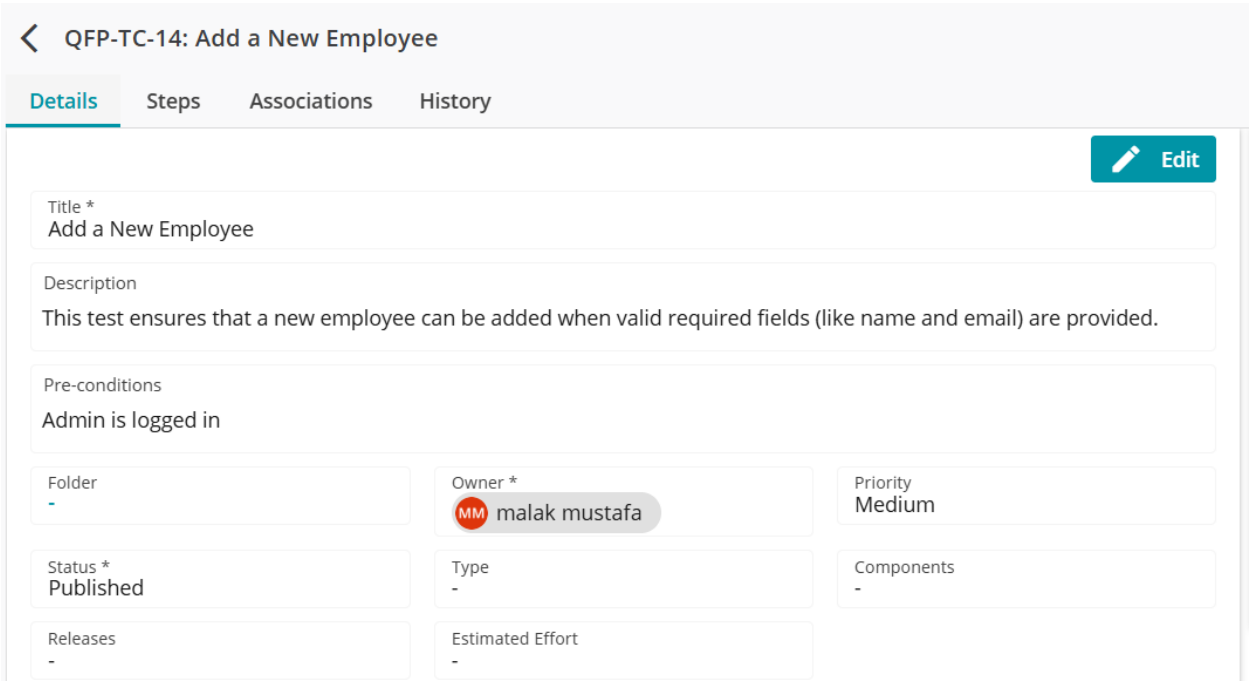
Expected result: A success message, "Invitation sent successfully," appears, and the invited user is listed.

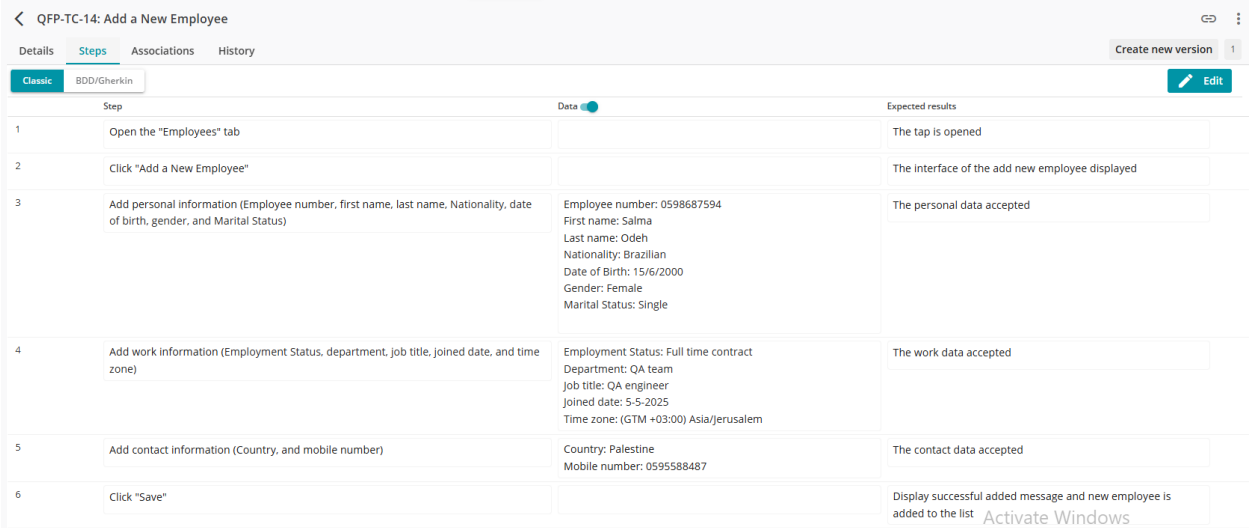
Actual result: That was an erroneous invitation.

Detected bug: The invitation didn’t work correctly.

Malak

TC-14: Add new employee (link in Jira)





Analyze the Result:

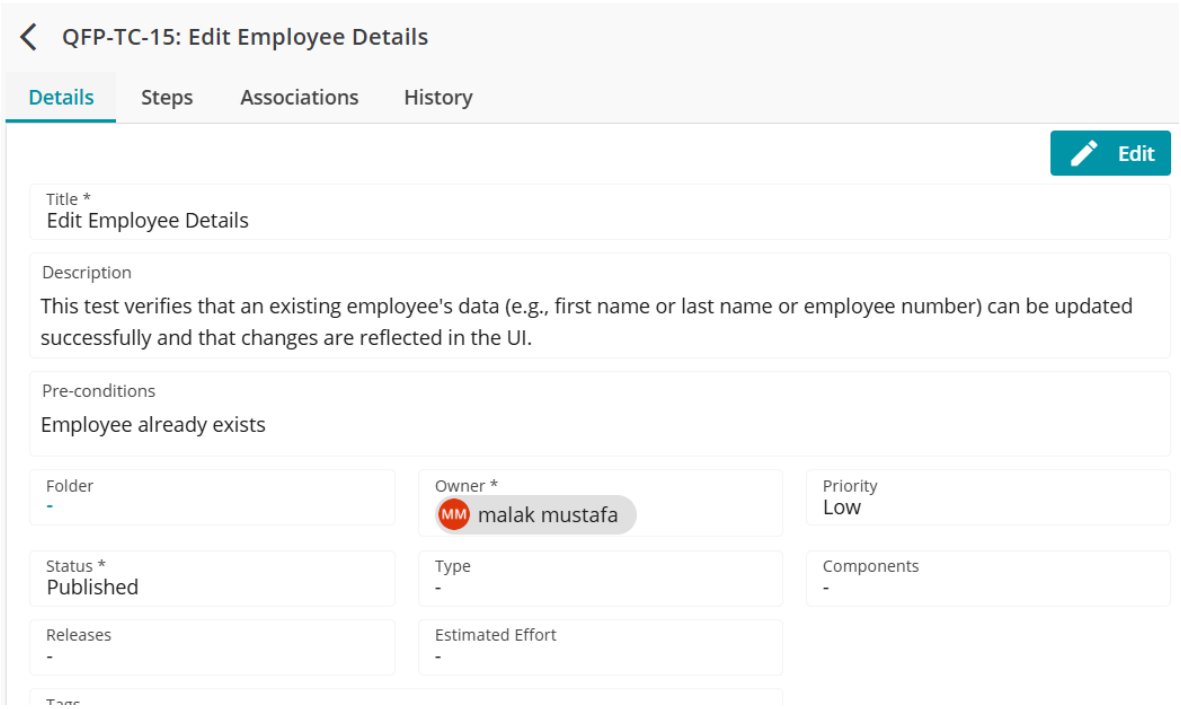
Expected result: New employee is added to the list, and displays the successful message “New employee profile created. Please find the new employee profile at the end of the list”.

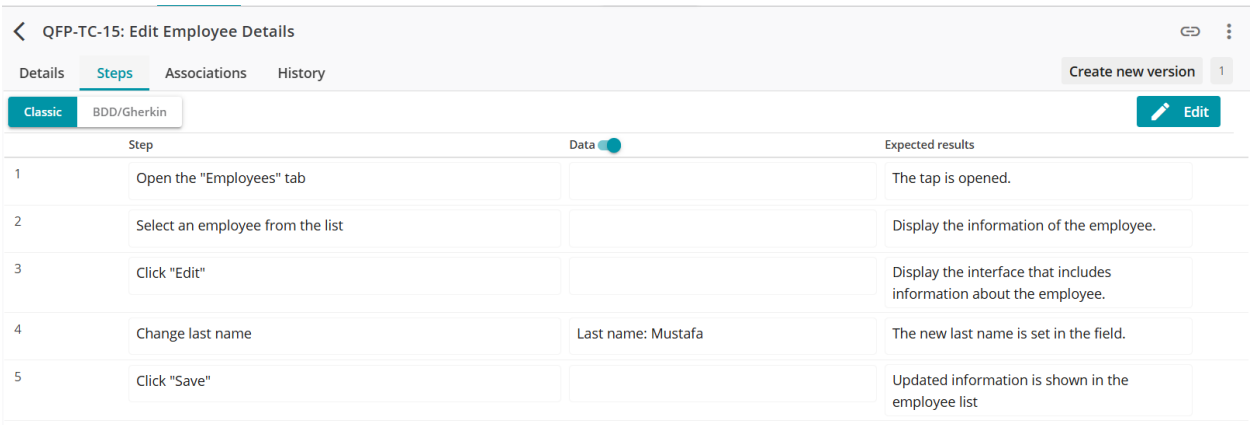
Actual result: The new employee is added, and the successful message is displayed.

Detected bug: No bug detected.

Malak

TC-15: Edit Employee Details (link in Jira)





Analyze the Result:

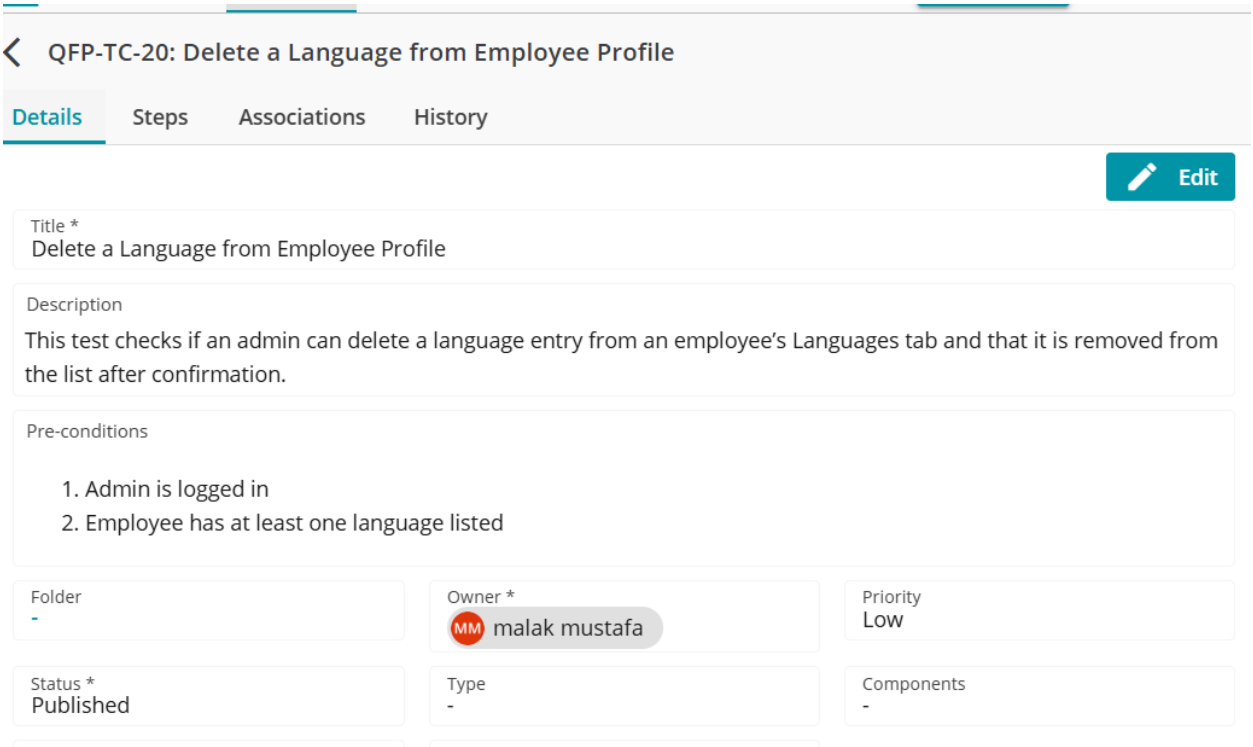
Expected result: Updated information is shown in the employee list

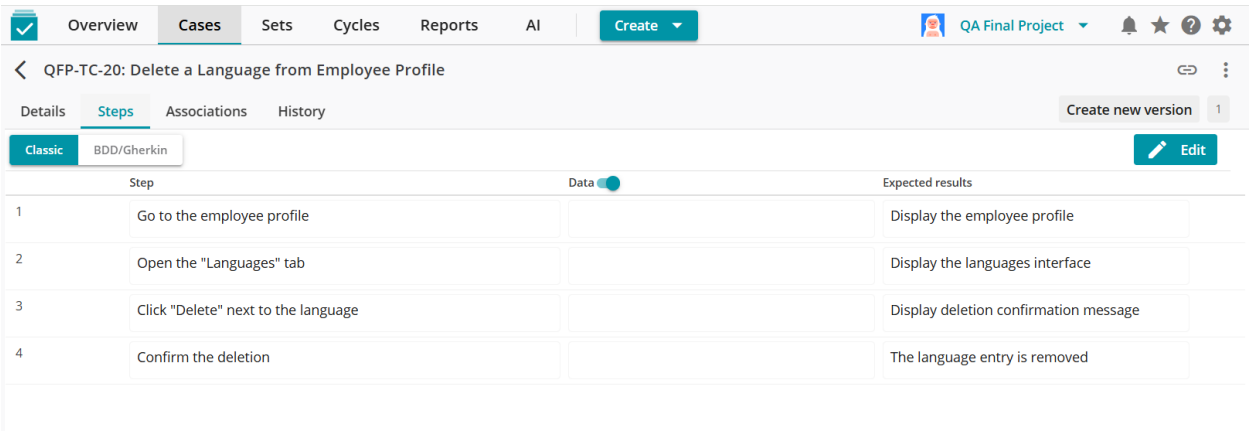
Actual result: The updated information is shown.

Detected bug: No bug detected.

Malak

TC-20: Delete a Language from Employee Profile (link in Jira)





Analyze the Result:

Expected result: The language entry is removed, and a confirmation message appears.

Actual result: The language was deleted.

Detected bug: No bug detected.

Summary of the Test Cases

Requirement ID	Requirement Description	Priority	Test Case	Test Result
R1	Daily Attendance Recording (Punch In / Out)	Low	TC-19: Create a previous timesheet	Pass
R2	View Attendance Reports	Medium	TC-18: Verify access as a supervisor	Pass
R3	Employee Profile Management	Medium	TC-14: Add new employee	Fail
R3	Employee Profile Management	Low	TC-15: Edit Employee Details	Pass
R3	Employee Profile Management	Low	TC-20: Delete language from profile	Pass
R4	Task Management	High	TC-01: Add Emergency Contact	Pass
R5	Edit Profile	Medium	TC-04: Edit password with only numbers	Pass

R6	Manage Document	High	TC-09: Add Document Type	Pass
R6	Manage Document	High	TC-11: Edit Document Type	Pass
R6	Manage Document	High	TC-12: Add Document Type with Empty Name	Pass
R7	Search, Filter	High	TC-10: Search Document Type	Pass
R8	Dashboard Overview	Medium	TC-17: Verify access supervisor	Pass
NF-R3	Secure platform + controlled access	Critical	TC-16: Access restriction for non-admin	Pass
NF-R5	Upload/retrieve documents quickly	High	TC-09: Add Document + upload details	Pass

White Box Testing

Layan

1. Statement coverage: is Direct Approval

```php
public function isDirectApproval($employeeId)
{
    $emp = new Employee();
    $emp->Load("id = ?", array($employeeId));
    if (empty($emp->approver1) && empty($emp->approver2) && empty($emp->approver3)) {
        return true;
    }
    return false;
}
```

Line Statement

2 Create new Employee() object

3 Load employee data by ID

Line Statement

4 if condition: check if approver1, approver2, and approver3 are all empty

5 Return true if no approvers

7 Return false otherwise

Test Cases:

Test Case	Scenario	Lines covered	Result
TC01	Employee has no approvals (approver1, approver2, approver3 are empty	2-3-4-5	true
TC02	Employee with at least one approver	2-3-7	false

2. Branch coverage: in updateApprovalStatus($type, $id, $currentEmployee, $status) function

Line #	Predicate	True	False
114	If (!$this->approvalChainExists($type, $id)) (Searches in approvalChainExist within the class with the type and ID inputs	Approval chain does not exist for this type and ID	Approval chain exists for this type and ID
118	If ($status != 0 && $status != 1) (Checks if the status is not valid, then displays the error message; if they are valid, then skips the if statement)	If status doesn't equal 0 or 1 (Invalid data)	If status equals 0 or 1 (skip the if condition)
129	Foreach ($eas as $ea) (Loops until the array is empty)	The array contains elements	The array is empty
129	If ($ea->status == 0) (Checks if the item's status in the array is 0 (Rejected)	If status equals 0	If status doesn't equal 0
132	elseif ($ea->active == 1) (checks if the item's status is active)	If active = 1	If active != 1

138	if ($level > 0) (If the item has a current level	Level > 0	The level is not above 0
144	if ($level < 3) (if the level doesn't reach the maximum level)	Level below 3	Level above or equal 3
148	if (empty($nextAL->id)) (Verify if next approver was loaded successfully)	If not loaded, sets the nextAL variable to null	If loaded successfully, keep the nextAL variable
154	if ($level > 0 && $currentEmployee != $currentAL->approver) (Validates if current employee is allowed to reject or approve)	If the current employee is not equal to the approver (return error)	If the current employee is equal to the approver (skip)
158	if (!empty($currentAL)) (Checks if the current approver was loaded)	If its loaded, update the status	If not loaded, skip
164	if (!empty($nextAL)) (Checks if there is a next approver to activate)	If there is an approver, activate it, and deactivate the others	If there is no approver, skip
177	if (!empty($currentAL)) (Checks if the current approver was loaded)	Reload the current approval	Skip

3. Path coverage: in updateApprovalStatus($type, $id, $currentEmployee, $status) function

All possible paths in the function:

The decision points in the function:

1. Does an Approval chain exist?

2. Is the status valid?

3. For each approval status:

- Rejected?
- Active?

4. Is level > 0?

5. Is level < 3?

6. Was nextAL loaded?

7. Is the current employee authorized?

8. Is currentAL set?

9. Is nextAL set?

10. Reload the current AL?

Path 1: (Accepted and next approver exists)

Approval chain exists for a specific type and ID -> status of the item = 1 -> all statuses for the type and ID are retrieved in $eas so its not empty -> no items rejected -> but have an active level -> the current approver is loaded -> the next approver is loaded -> the current employee == current AL -> current AL is update -> next AL is activated -> current AL is refreshed.

Path numbers: 122 -> 123 -> 124 -> 126 -> 127 -> 129 -> 132 -> 133 -> 137 -> 138 -> 139 -> 140 -> 143 -> 145 -> 146 -> 158 -> 160 -> 161 -> 164 -> 177 -> 183

Path 2: Rejection exists:

Approval chain exists for a specific type and ID -> status is valid -> all statuses for the type and ID are retrieved in $eas, so it's not empty -> rejection is detected in the item -> returns error message: item is already rejected

122 -> 123 -> 124 -> 126 -> 127 -> 129 -> 130

Path 3: Invalid status:

Approval chain exists for a specific type and ID -> status is invalid -> returns error message: Invalid data

118 -> 119

Path 4: Employee not authorized

Approval chain exists for a specific type and ID -> status is valid -> all statuses for the type and ID are retrieved in $eas so its not empty -> no items rejected -> but have an active level -> the current approver is loaded -> the next approver is loaded -> the current employee is not current AL -> returns error message: you're not allowed to approve or reject.

122 -> 123 -> 124 -> 126 -> 127 -> 129 -> 132 -> 133 -> 137 -> 138 -> 143 -> 144 -> 154

Path 5: No next approver

Approval chain exists for a specific type and ID -> status of the item = 1 -> all statuses for the type and ID are retrieved in $eas so its not empty -> no items rejected -> but have an active level -> the current approver is loaded -> No next approver exists -> the current employee == current AL -> current AL is update -> current AL is refreshed.

122 -> 123 -> 124 -> 126 -> 127 -> 129 -> 132 -> 137 -> 138 -> 148 -> 158 -> 177 -> 183

Path 6: All statuses for the type and ID are not retrieved ($eas is empty):

Approval chain exists for a specific type and ID -> status of the item = 1 -> all statuses for the type and ID are not retrieved in $eas is empty -> the current approver is loaded -> the next approver is loaded -> the current employee == current AL -> current AL is update -> next AL is activated -> current AL is refreshed.

122 -> 137 -> 138 -> 143 -> 144 -> 158 -> 164 -> 177 -> 183

Rawa

getDocumentTypesForUser() Method

Logic Overview:

The method getDocumentTypesForUser() returns document types based on the current user's role. The access is filtered as follows:

```
    public function getDocumentTypesForUser()
    {
        $documents = new Document();
        if (BaseService::getInstance()->currentUser->user_level === 'Employee'
            || BaseService::getInstance()->currentUser->user_level === 'Restricted Employee
        ) {
            $documents = $documents->Find('share_with_employee = ?', ['Yes']);
        } else {
            $documents = $documents->Find('1 = 1');
        }

        return $documents;
    }
}
```

White-box Testing Techniques Applied:

1. Statement Coverage

To achieve **100% statement coverage**, each line of code must be executed at least once.

- **Test Case 1:** user_level = "Employee"
  → Ensures execution of: Find('share_with_employee = ?', ['Yes'])

- **Test Case 2:** user_level = "Manager"
  → Ensures execution of: Find('1 = 1')
- All statements are executed at least once → **Full statement coverage achieved**.

---

2. Branch Coverage

Branch coverage requires testing all possible outcomes of each decision point.

- **Condition:** if (A || B)
    - **Test Case 1:** user_level = "Employee" → condition is **true**

- **Test Case 2:** user_level = "Restricted Employee" → condition is **true**
- **Test Case 3:** user_level = "Admin" → condition is **false**

All branches (true and false) are tested → **Full branch coverage achieved**.

---

3. Path Coverage

There are two distinct logical paths:

- **Path 1:** User is “Employee” or “Restricted Employee”
  → Executes: Find('share_with_employee = ?', ['Yes'])
- **Path 2:** User is any other role (e.g., “Manager”, “Admin”)
  → Executes: Find('1 = 1')
- **Test Case 1:** Covers Path 1
- **Test Case 2:** Covers Path 2

All paths are exercised → **Full path coverage achieved**.

Summary:

The function getDocumentTypesForUser() has been fully tested using white-box techniques. All conditions, branches, and execution paths are covered using well-defined test cases based on user roles.

Nahed

```
public function listEmployeeAttendance(User $user, $parameter)
{

    if ($user->user_level !== 'Admin' && $user->employee != $parameter) {
        $employee = new Employee();
        $employee->Load('id = ?', [$parameter]);
        if ($employee->supervisor != $user->employee) {
            return new IceResponse(
                IceResponse::ERROR,
                self::RESPONSE_ERR_PERMISSION_DENIED,
                401
            );
        }
    }

    $query = new DataQuery('Attendance');
    $query->addColumn('id');
    $query->addColumn('employee');
    $query->addColumn('in_time');
    $query->addColumn('out_time');
    $query->addColumn('note');

    $query->setOrderBy('in_time desc');

    $limit = self::DEFAULT_LIMIT;
    if (isset($_GET['limit']) && intval($_GET['limit']) > 0) {
        $limit = intval($_GET['limit']);
    }
    $query->setLength($limit);

    $query->addFilter(new Filter('employee', $parameter));

    return $this->listByQuery($query);
}
```

Statement Coverage

Line #	Statement	Expected Result	Actual result	Status	Test Case
3	if ($user->user_level !== 'Admin' && $user->employee != $parameter)	Do not Display TimeSheet	Do not Display TimeSheet	Pass	TC-16
4	new Employee()	New Employee	New Employee	pass	
5	$employee->Load(...)	Loading data(id,parameter)	Loading data(id,parameter)	pass	
6	if ($employee->supervisor != $user->employee)	Do not Display TimeSheet	Do not Display TimeSheet	pass	TC-17
7	return new IceResponse(...)	new IceResponse(...)	new IceResponse(...)	pass	
19	return $this->listByQuery($query)	Retun TimeSheet	Retun TimeSheet	pass	TC-18

Branch Coverage

Line #	Predicate (Condition)	True in Test Case	False in Test Case
3	$user->user_level !== 'Admin'	User level==’Admim’ return listByQuery($query)	User!==’Admin’ return ERROR
3	$user->employee != $parameter	User==$parameter return listByQuery($query)	User!=='$parameter return ERROR
6	$employee->supervisor != $user->employee	$user->supervisor->$employee return listByQuery($query)	$usert!= supervisor->$employee return ERROR
12	isset($_GET['limit']) && intval(...) > 0	$_GET['limit']<0 retutn DEFULT-LIMIT	$_GET['limit']>0 $limit=$_GET['limit']

Path Coverage

Path1→User is Admin→Return Query.

Path2→User is not Admin And is Employee→Return Query

Path3→User is not Admin but he is Supervisor→Return Query

Path4→User is not Admin and He is not the Employee→Return Error

Malak

The code of the function getTaskCreators()

```php
public function getTasksCreators()
{
    $taskCreators = [];

    $user = BaseService::getInstance()->getCurrentUser();

    if ('Admin' === $user->user_level) {
        $taskCreators[] = function () {
            $employee = new Employee();
            $employee->Load('employee_id = ?', ['EMP001']);
            if ($employee->first_name === 'IceHrm') {
                return (new Task(Task::PRIORITY_ERROR, 'Default employee has not been removed or modified'))
                    ->setLink(
                        CLIENT_BASE_URL.'?g=admin&n=employees&m=admin_Employees',
                        'Edit Employees'
                    );
            }

            return null;
        };
    }

    $employeeId = $user->employee;
    if (!empty($employeeId)) {
        $taskCreators[] = function () use ($employeeId) {

            $employeeState = new EmployeeStatus();
            $employeeState->Load('employee = ? and status_date = ?', [ $employeeId, CalendarTools::getServerDate()]);
            if (empty($employeeState->id)) {
                return (new Task(Task::PRIORITY_TOP, 'You haven\'t set your status for the day.'))
                    ->setLink(
                        CLIENT_BASE_URL.'?g=modules&n=dashboard&m=module_Personal_Information',
                        'Set your Status and Goals'
                    );
            }

            return null:
```

Statement Test

Test Case ID	Description	Input Condition
TC1	User is Admin, employee “EMP001” has first name “IceHrm”	user_level = “Admin”, employee_id = “EMP001”, first_name = “IceHrm”
TC2	User is Admin, employee “EMP001” has first name not equal to “IceHrm”	user_level = “Admin”, employee_id = “EMP001”, first_name = “Malak”
TC3	User is NOT Admin but has a valid employee ID, no status set today	user_level =”Employee”, employeeId = valid, No record in EmployeeStatus table
TC4	User is not an Admin but has a valid employee ID with status already set	user_level = “Employee”, employeeId = valid, status exists
TC5	User is not an Admin and has no employee ID	user_level =” Employee”, employeeId = null or “ “

Branch Coverage

Line #	Predicate	True	False
21	if ('Admin' === $user ->user_level)	user_level = 'Admin'	user_level = 'Manager'
25	if ($employee->first_name === 'IceHrm')	first_name = 'IceHrm'	first_name = 'Ali'
38	if(!empty($employeeId))	Employeeid= 1234	Employeeid= null or " "
43	if(empty($employeeState->id))	employeeStatusid = null	employeeStatusid = 567

Path Convergence

Path1 → user is admin → employee ID = EMP001 → first name is "IceHrm" → task created

Path2 → user is Admin → employee ID = EMP001 → first name is not 'IceHrm' → no task → returned

Path3 → user is not Admin → employee ID is set → employee status for today is already set → no task returned

Path 4 → user is not Admin → employee ID is set → employee status for today is not set → task created

Path 5 → user is not Admin → employee ID is empty → no task created

Automated Test In Katalon.
Edit the employee name in the languages tab

Test Automation

This section contains automated test cases using Katalon. Each group member implemented 3 test cases, ensuring efficient and reliable testing processes.

Layan

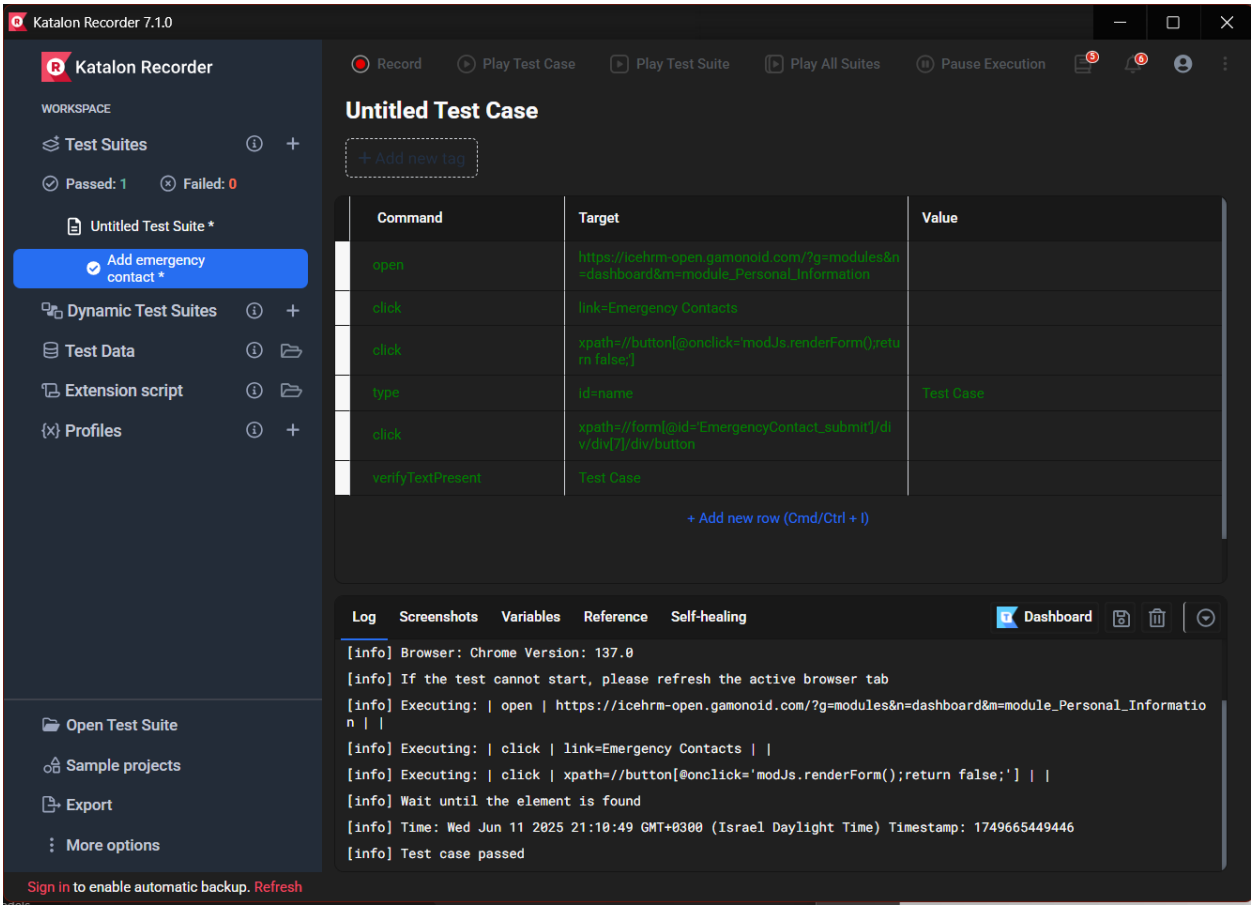
Test Case 1:

This test case verifies that a new emergency contact can be added successfully by entering at least the name.

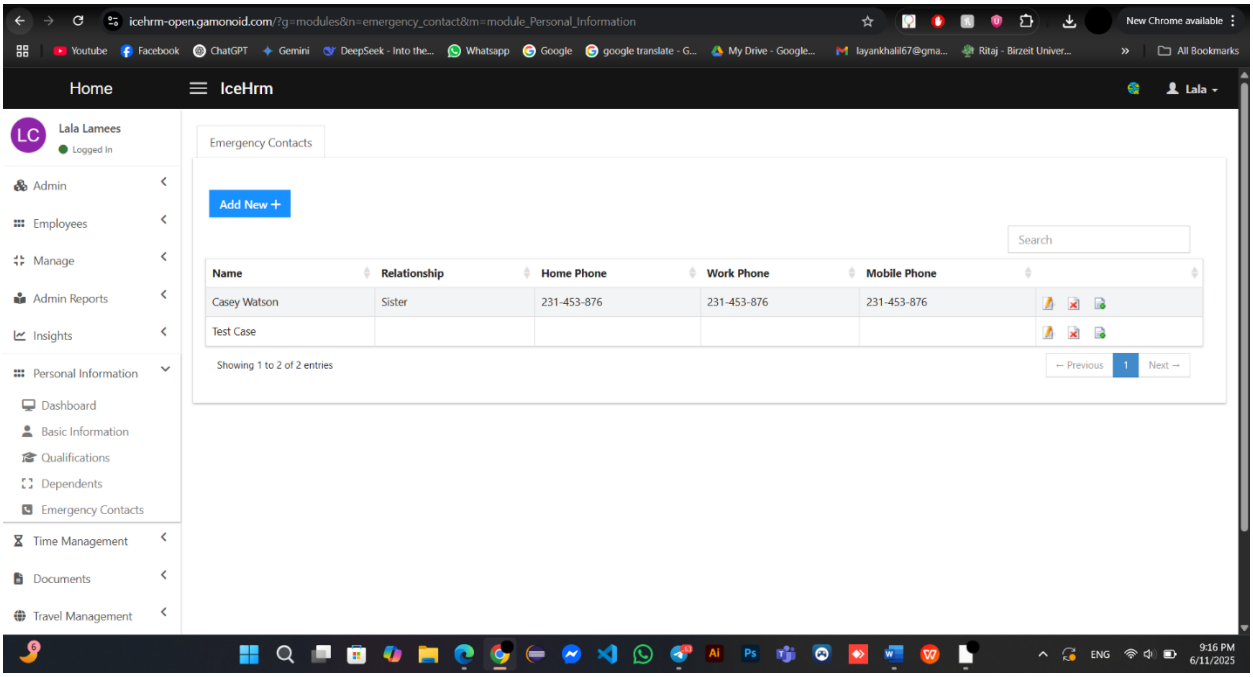
Steps executed automatically: Open personal information -> Emergency contacts -> Add new -> Enter name -> Submit

Expected result: The contact “Test Case” appears in the list.

Actual Result: Test passed successfully.



The name was added successfully to the contact list:



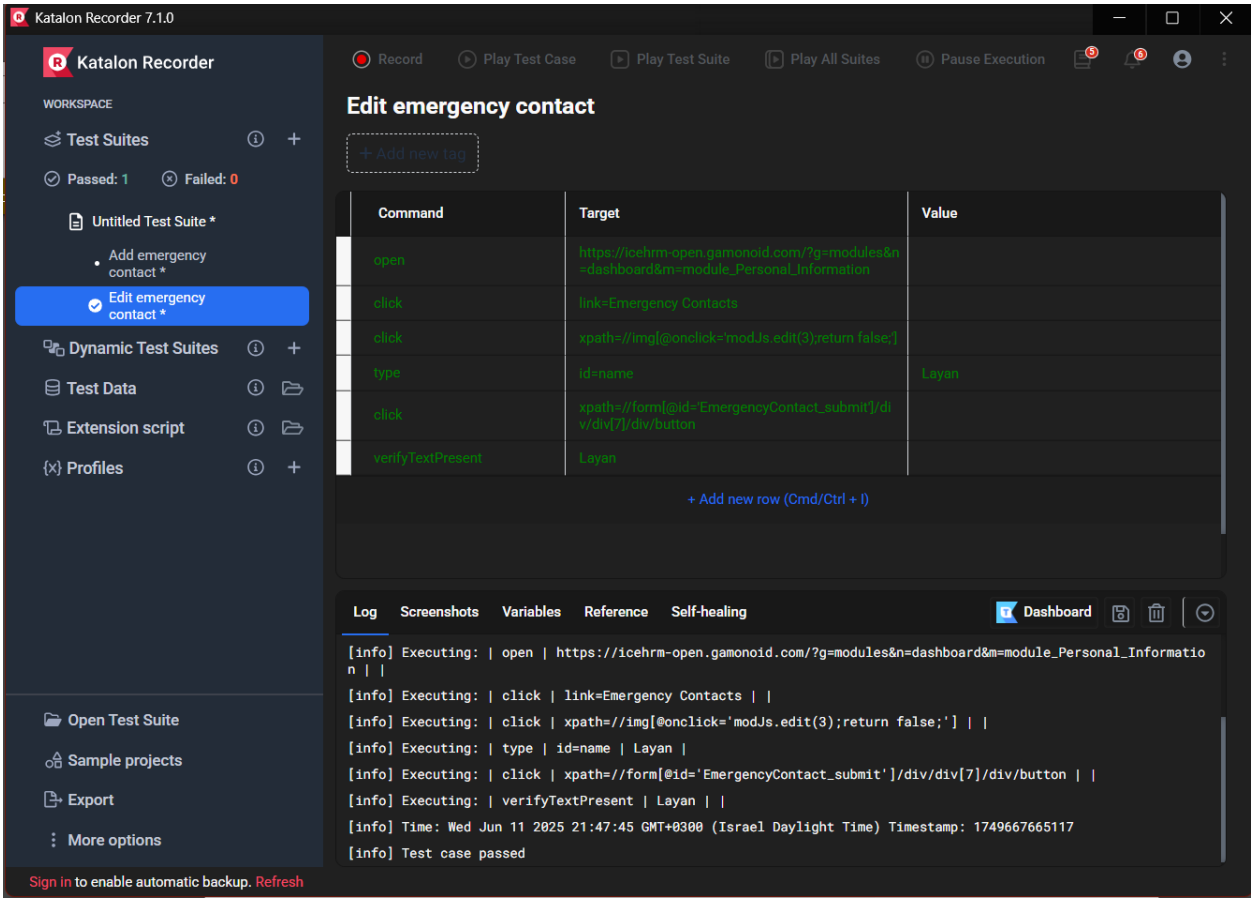
Test Case 2:

This test case verifies that any emergency contact can be editable.

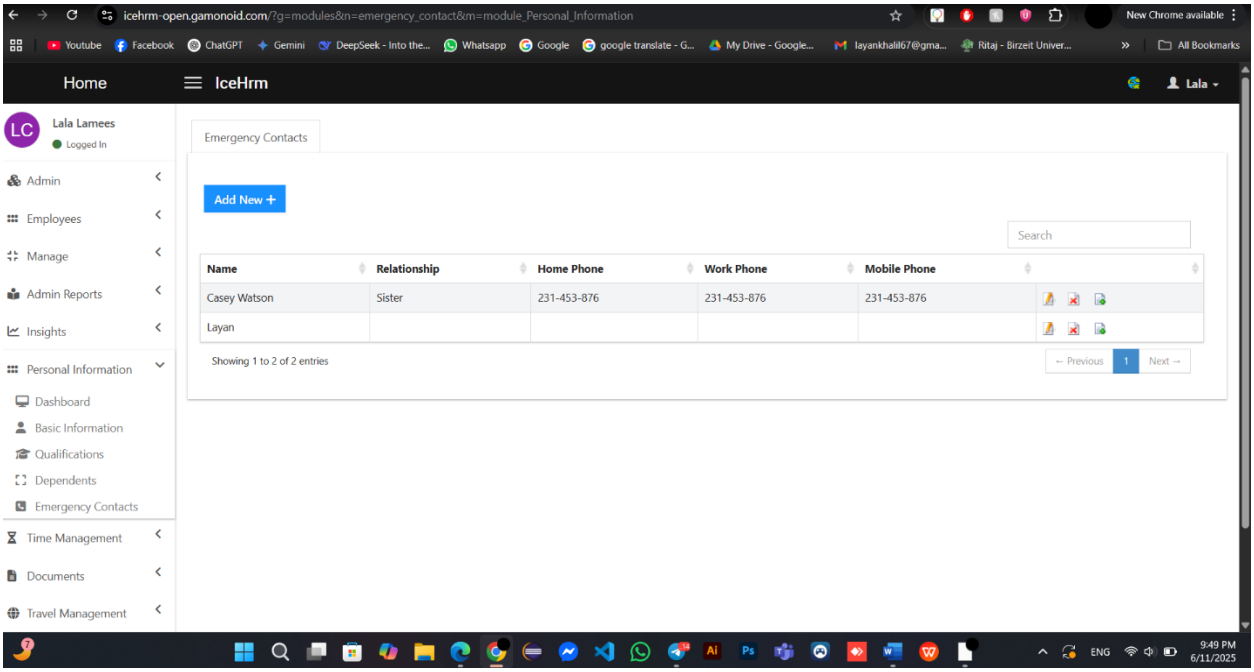
Steps executed automatically: Open personal information -> Emergency contacts -> Edit -> Edit name -> submit

Expected result: The contact “Test Case” is edited to “Layan”.

Actual Result: Test passed successfully.



The name was edited successfully



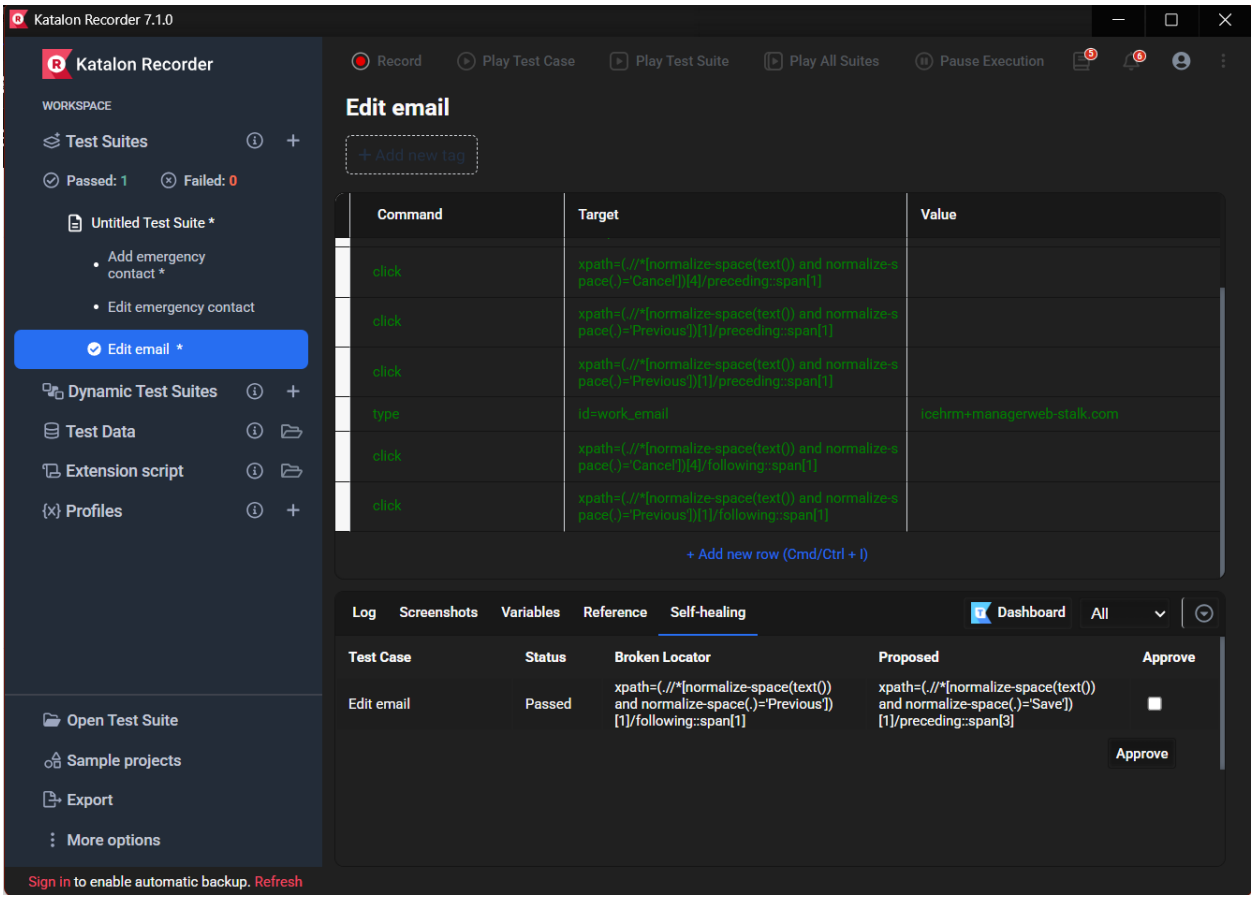
Test Case 3:

This test case verifies that the email cannot be editable to a wrong email.

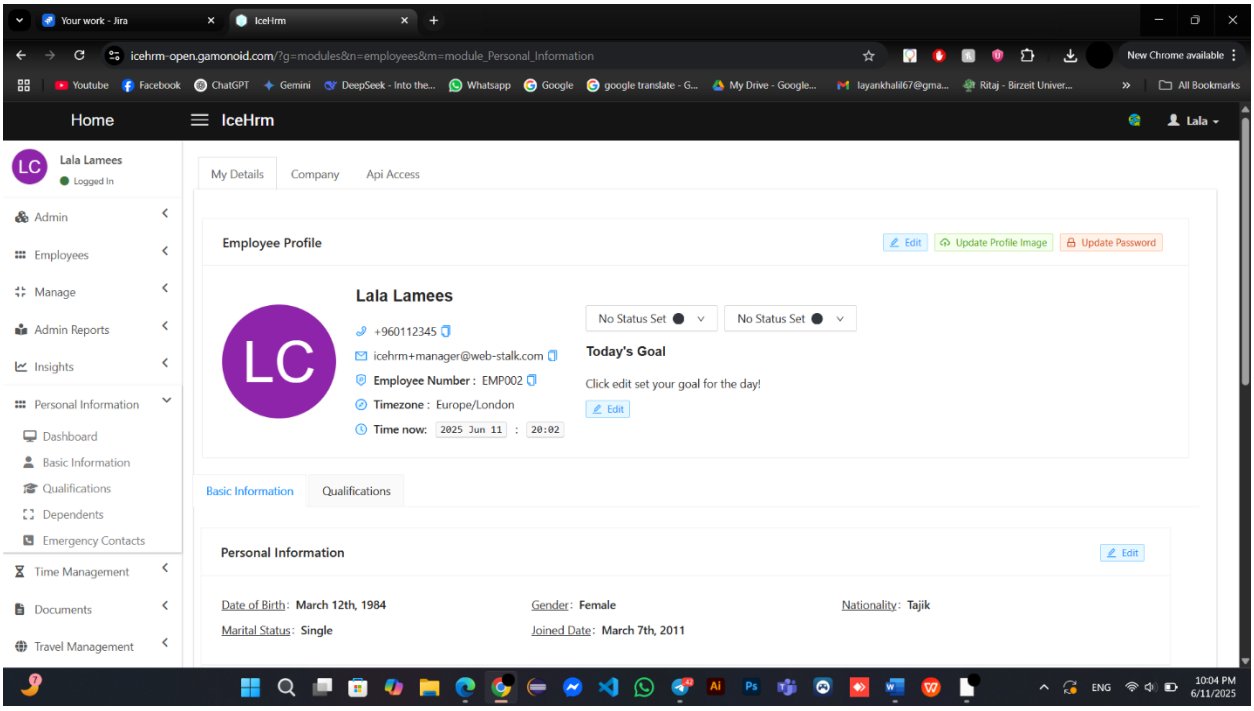
Steps executed automatically: Open personal information -> Basic information -> Edit -> Contact -> Edit the email -> Save -> Cancel

Expected result: The system displays a validation error message and prevents saving the invalid email.

Actual Result: Test passed successfully.



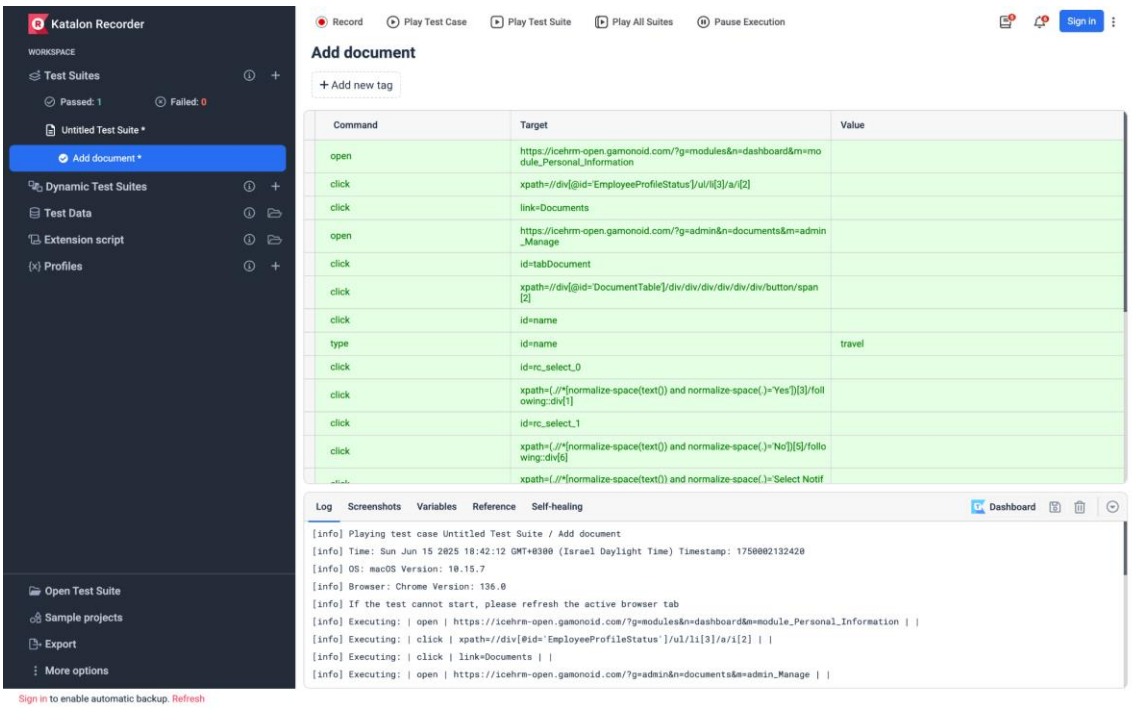
the system displayed an error message and could not save the edited email.

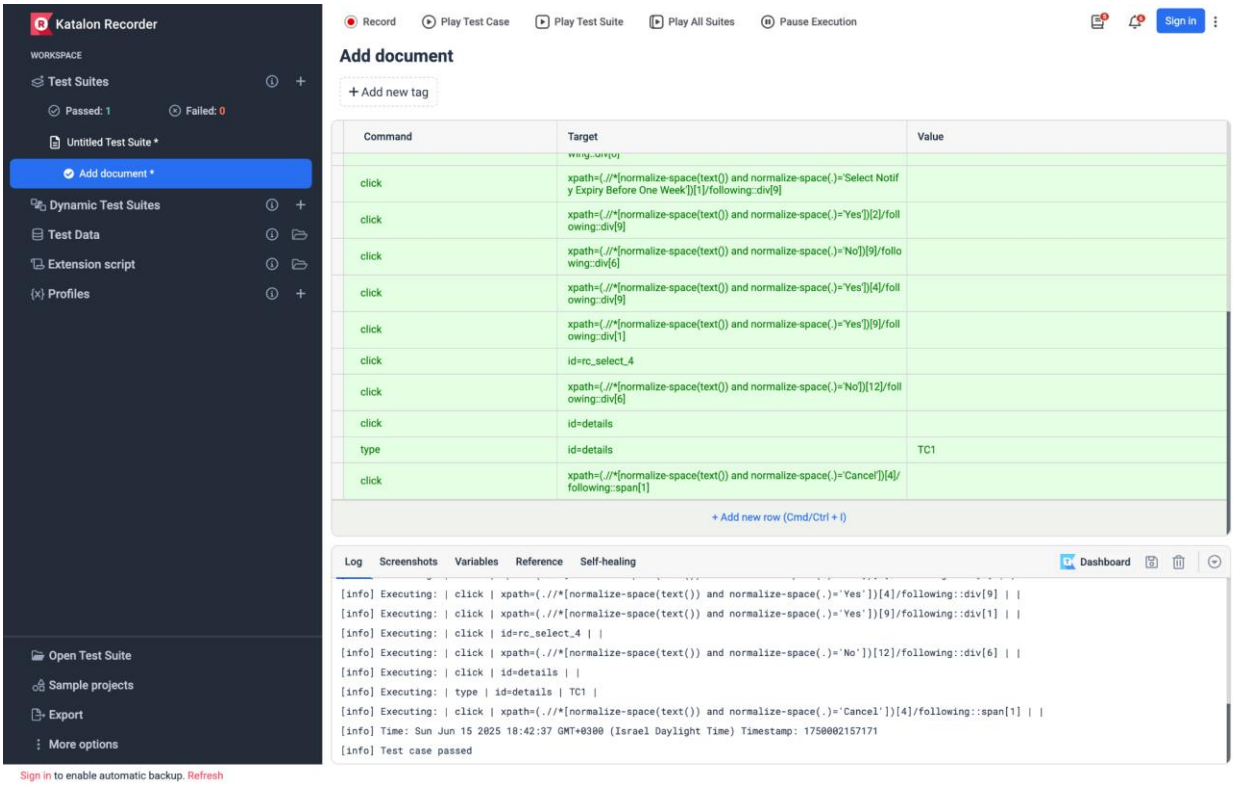


Rawa

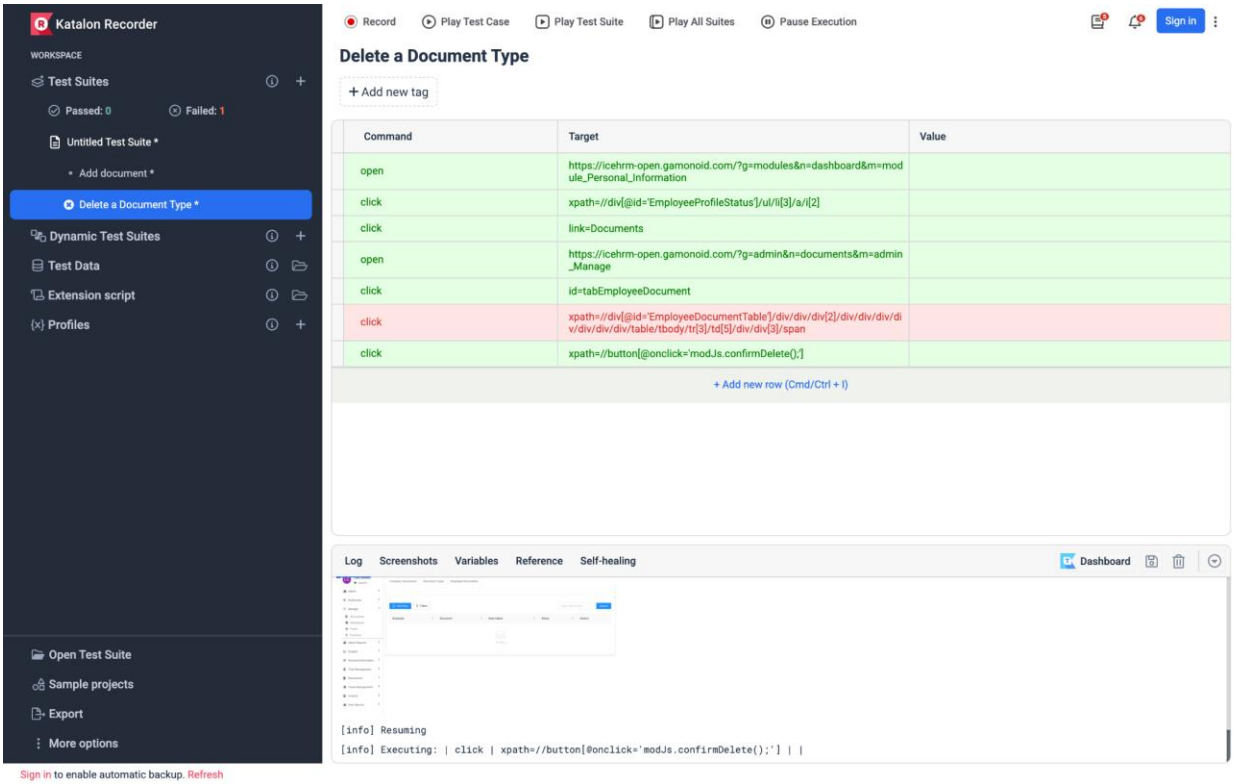
1- Add a new Document Type

Purpose: To verify that a new document type can be added successfully.

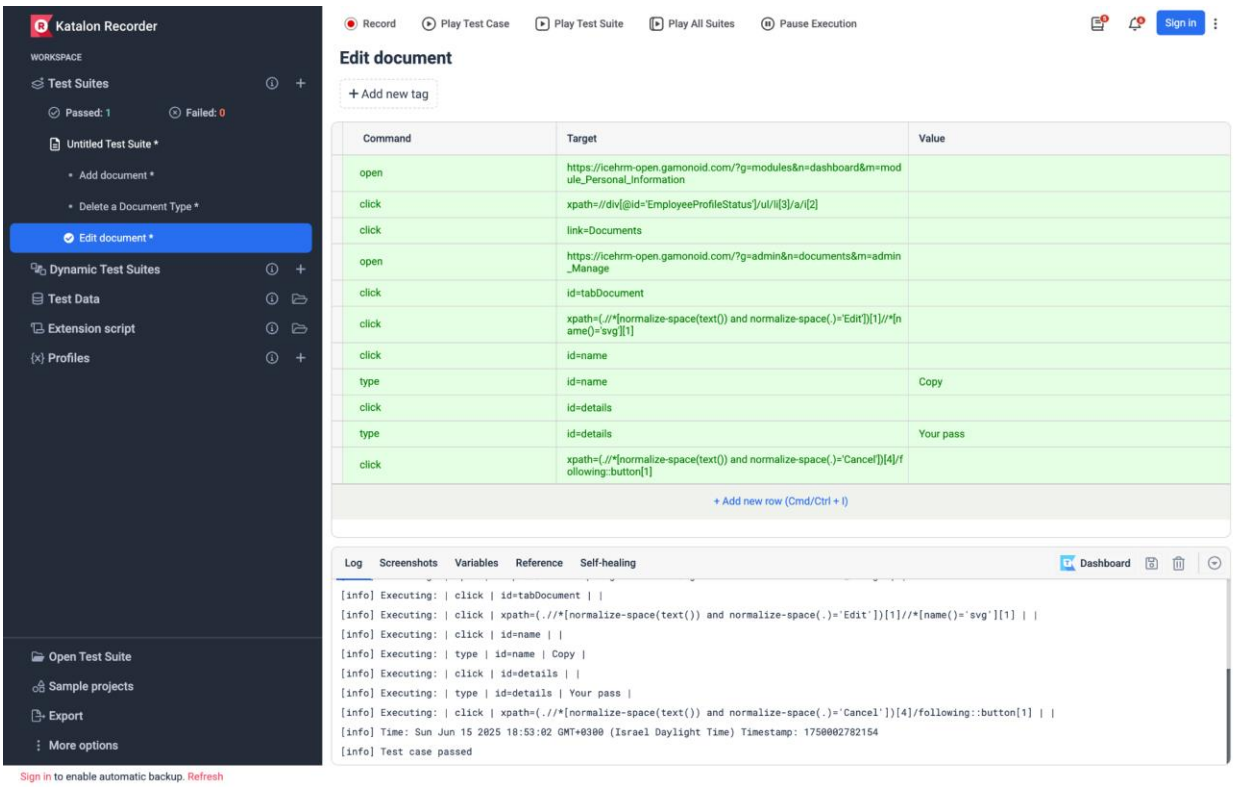




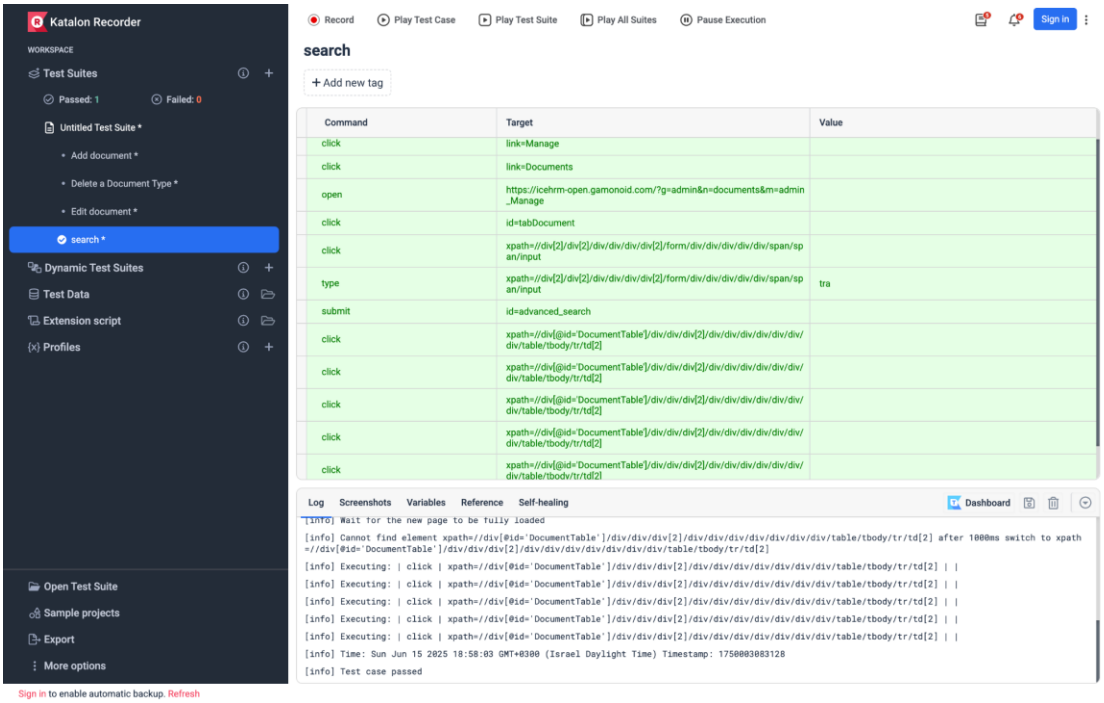
2- Delete a Document Type



3- Edit Document Type

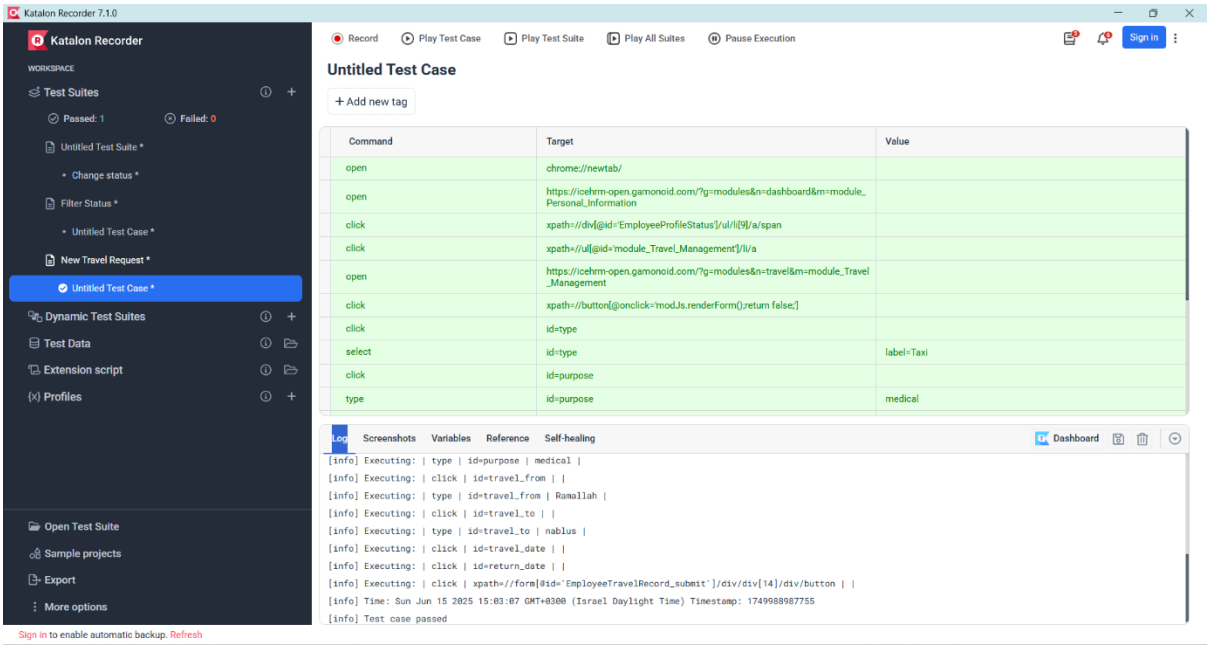


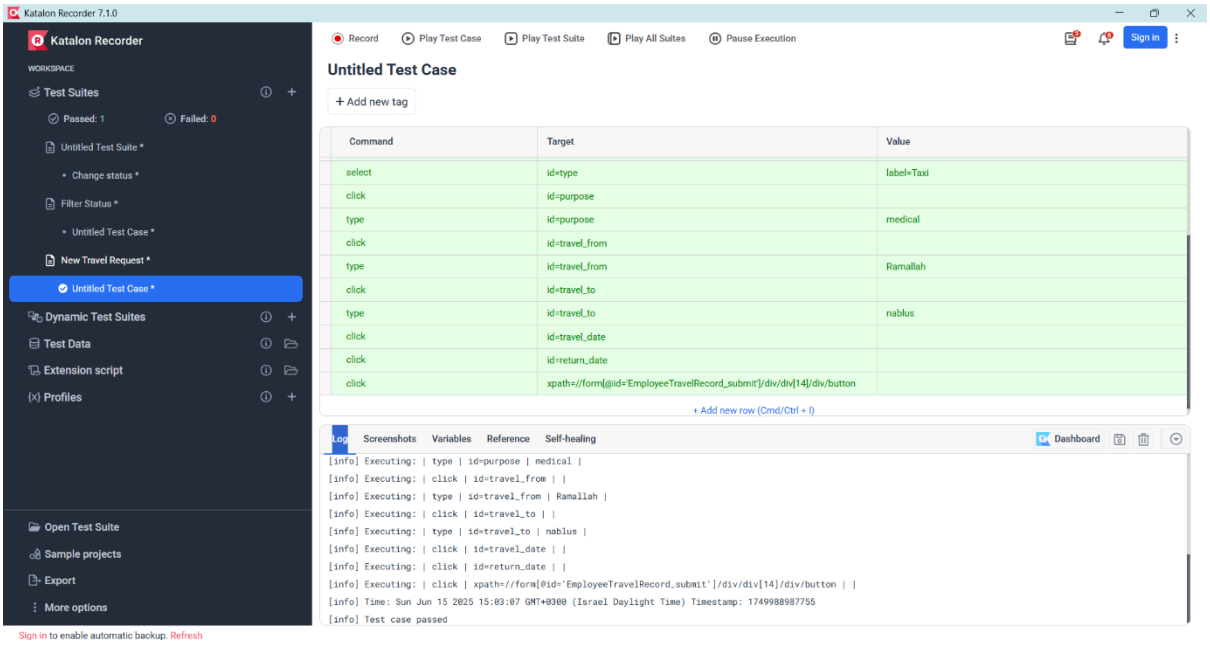
4- Search functionality verification



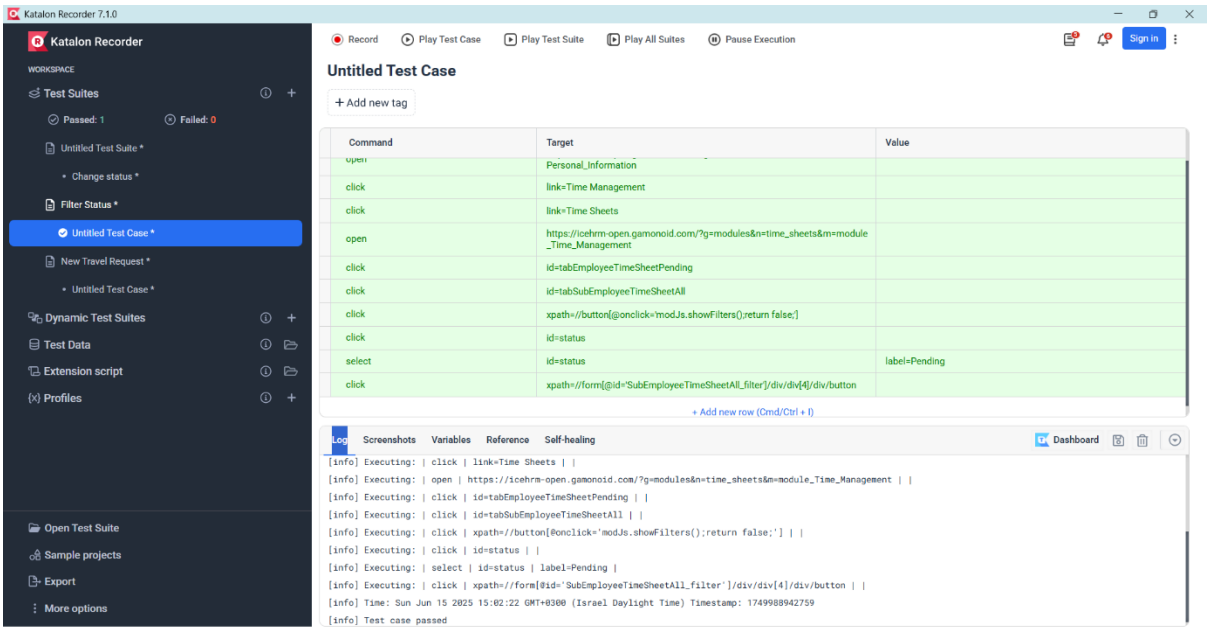
Nahed

Travel Form validation

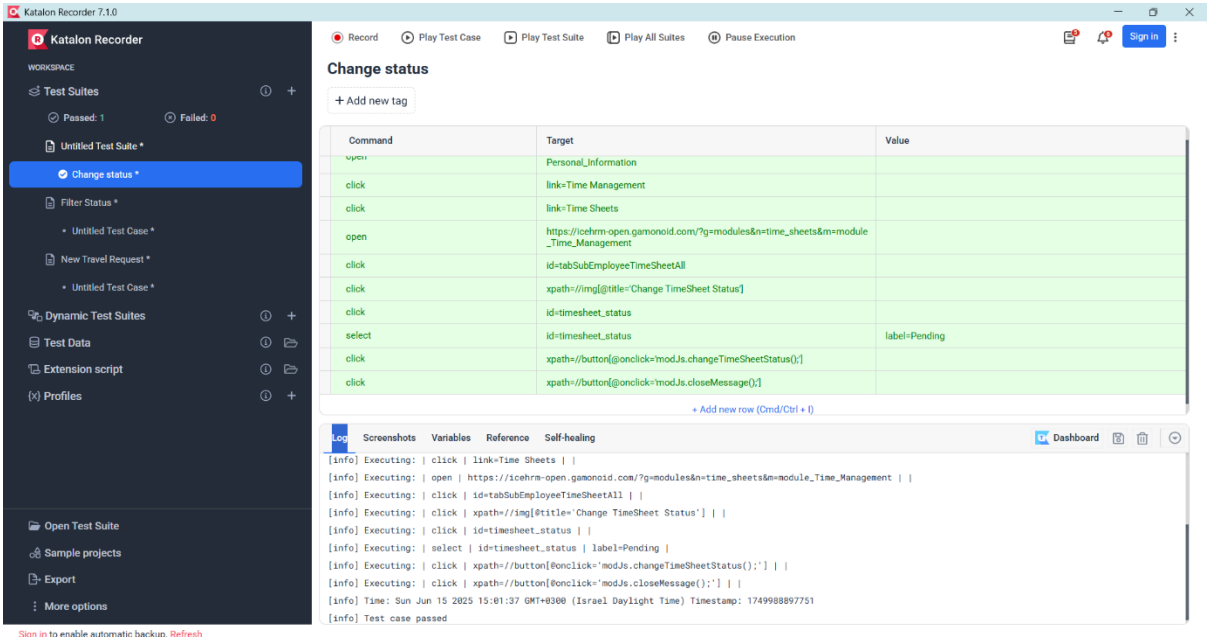




Filter by timesheet status

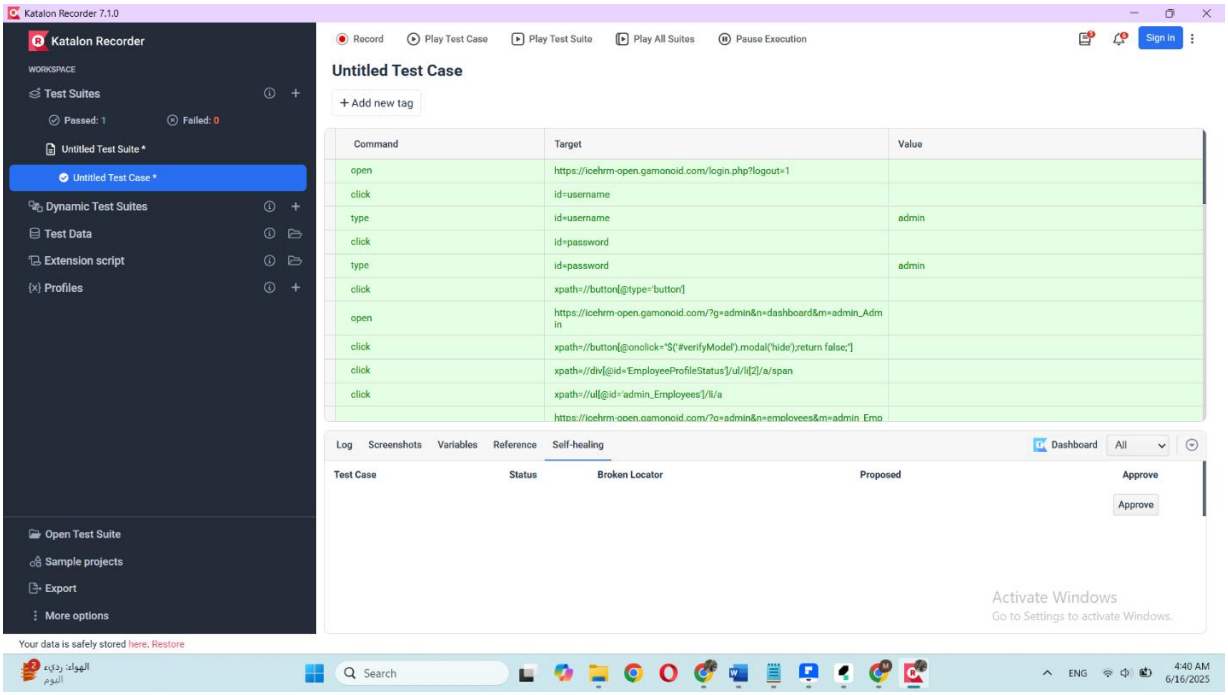


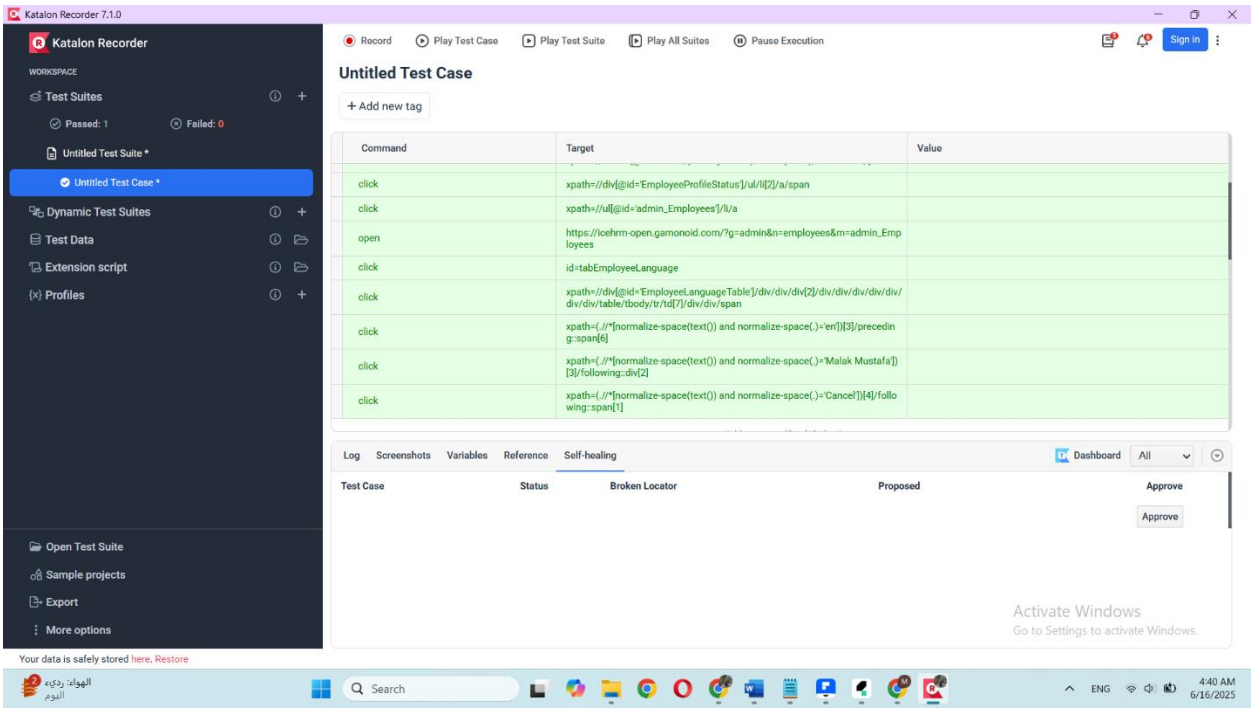
Change Timesheet status



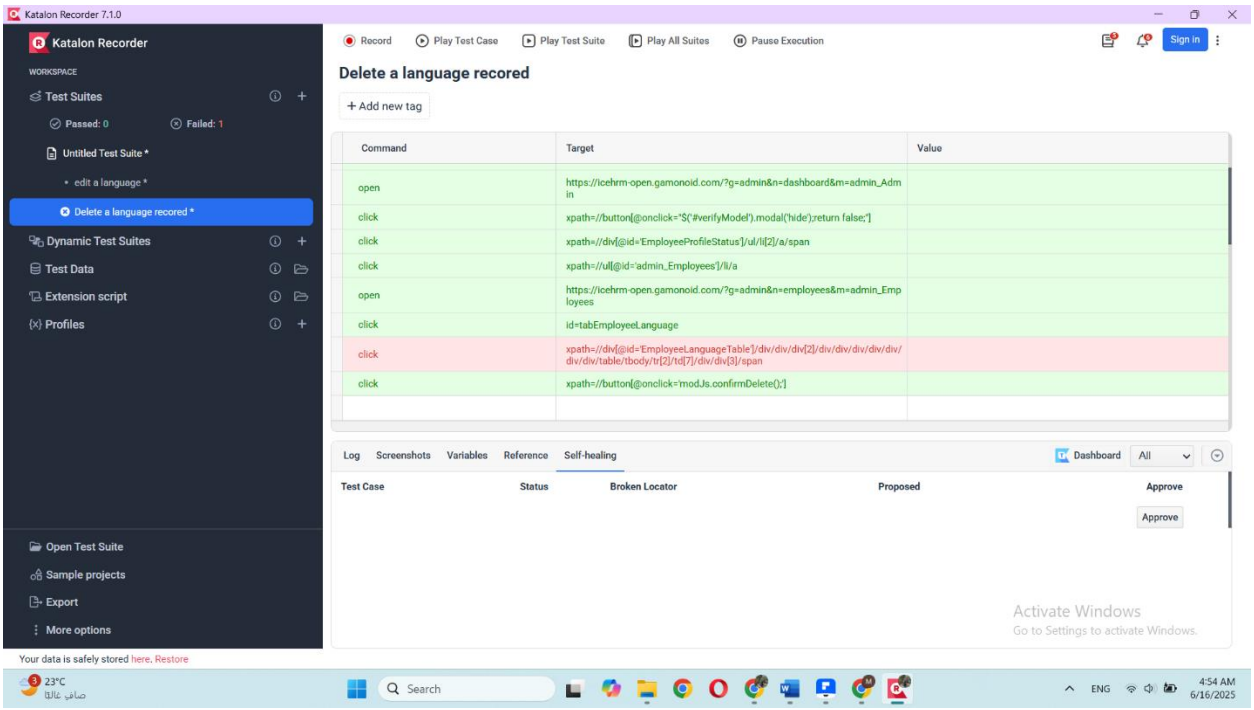
Malak

Edit a Language

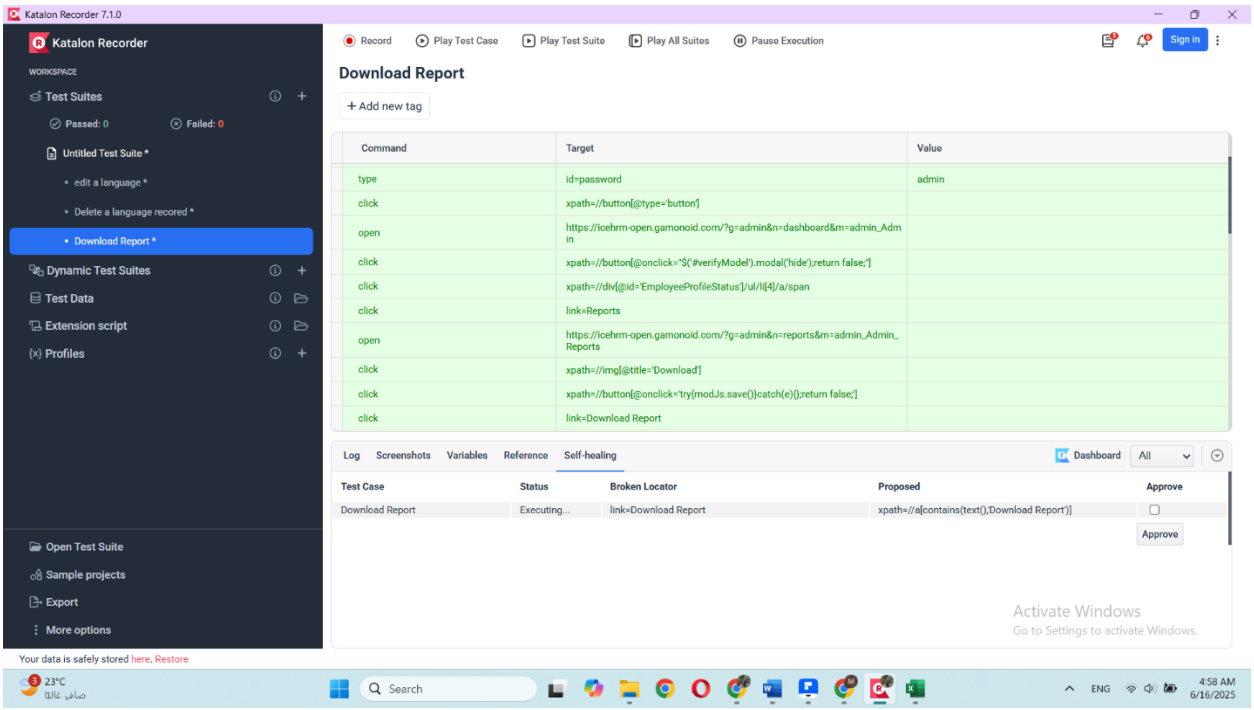




Delete a language record



Download a report



Performance & Load Testing

This section includes the performance and load testing conducted for the IceHRM Document Management Module, using Apache JMeter to evaluate system behavior under varying levels of concurrent user activity. The goal is to simulate real-world conditions and assess the reliability and scalability of the system.

Scenario: Load Testing

The objective of this test was to measure the performance of the login pageunder expected user load, ensuring that the system can maintain acceptable response times without performance degradation.

Configuration:

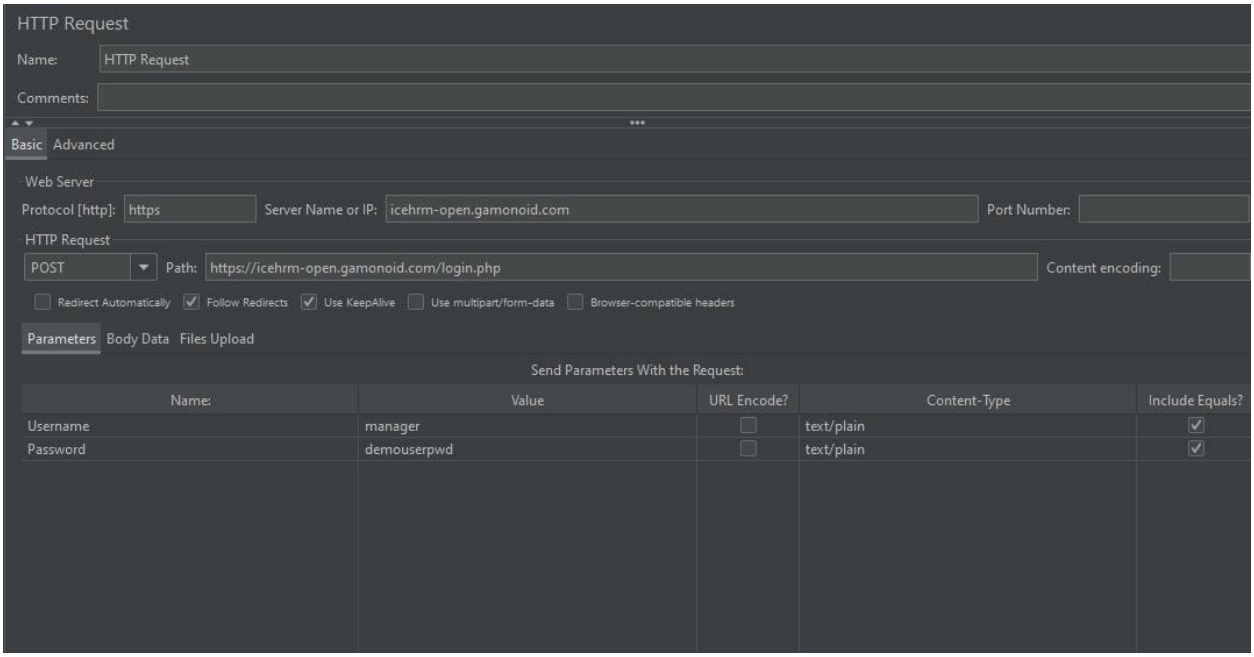


Figure: http request (The site requires login.)

Thread Properties:

- Number of Users (Threads): 300
- Ramp-Up Period (seconds): 2
- Loop Count: 1

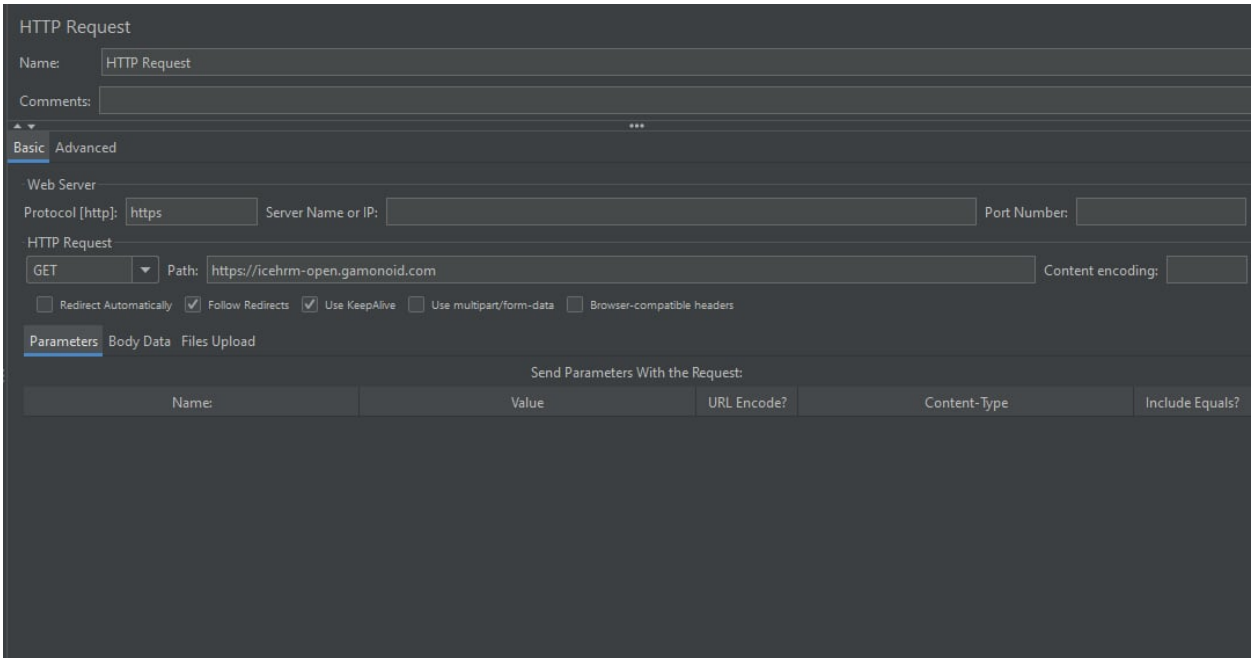


Figure: http request

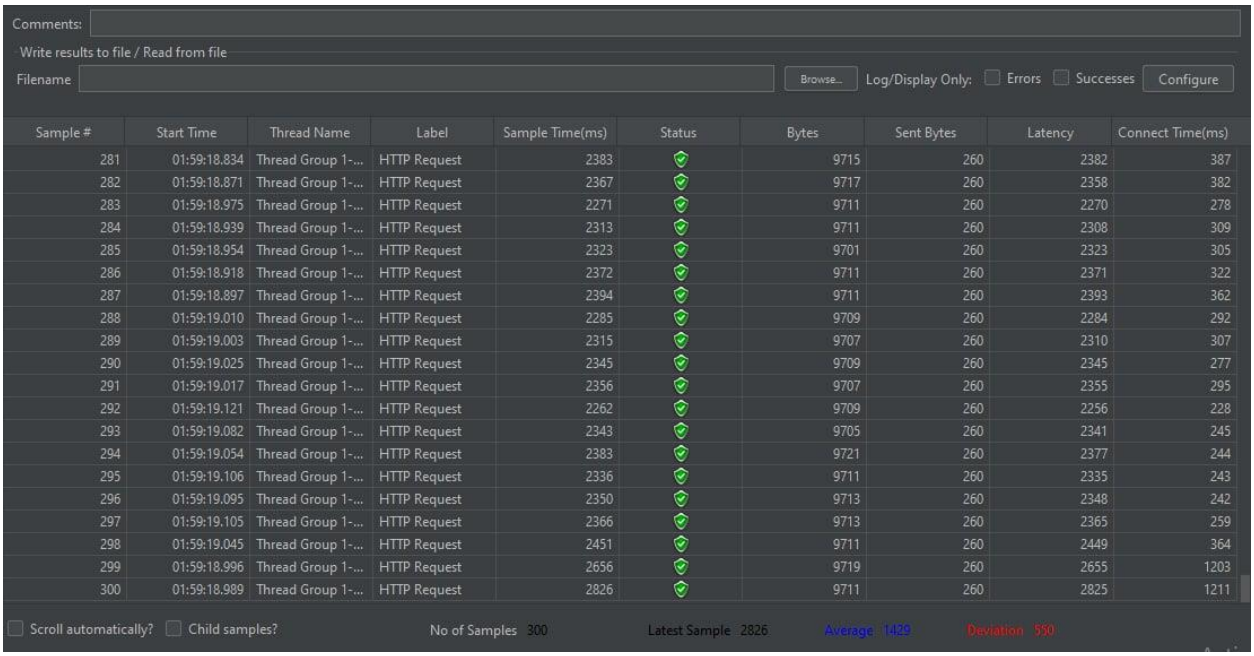


Figure: view result table for 300 users

Load testing: measure the IceHrm performance to **the manage page** under expected loads to ensure it can handle the anticipated user base without degrading performance. **Thread properties:**

- Number of users: 300
- Ramp-up period (in sec): 2
- loop count: 1

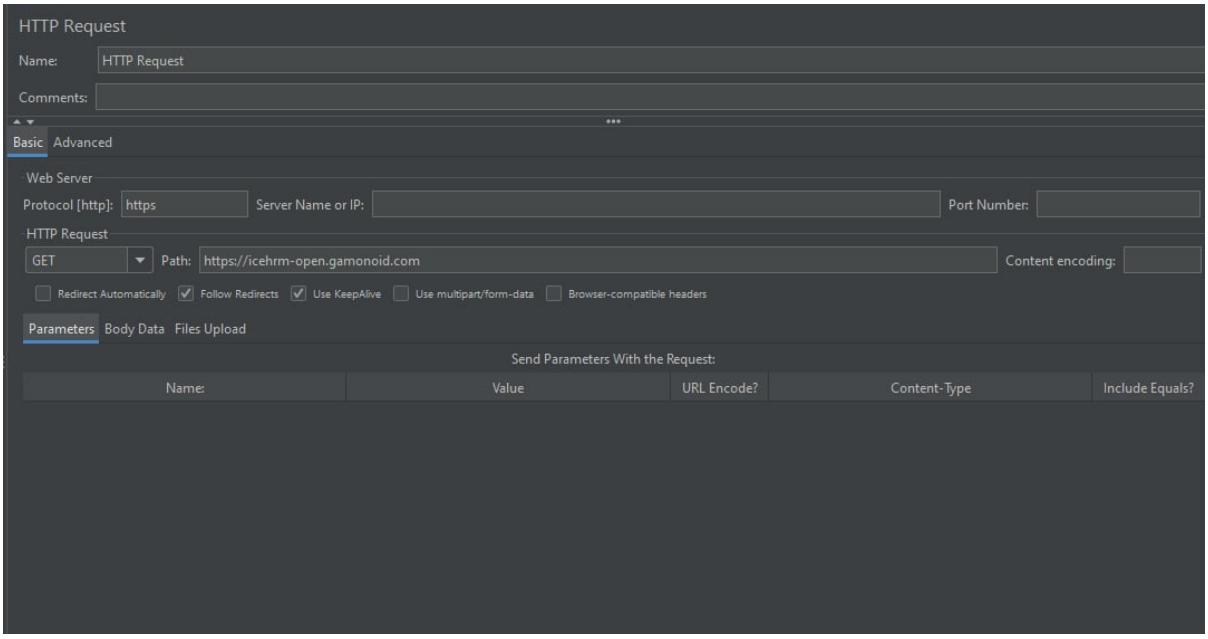
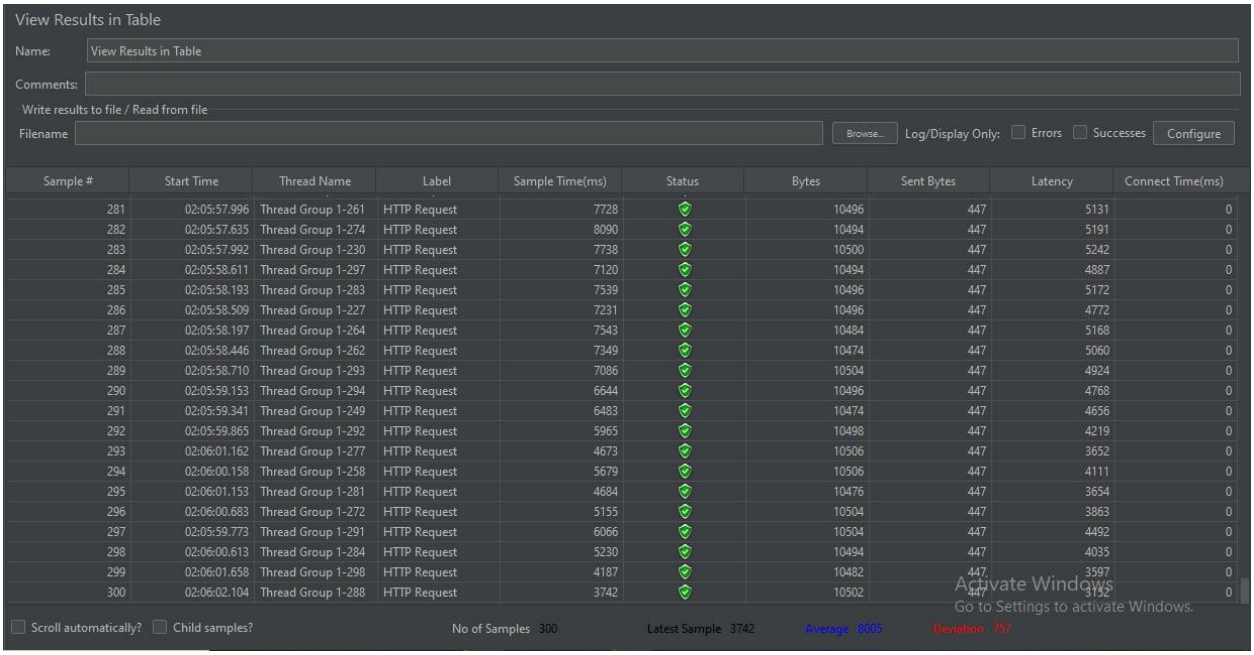


Figure: http Request



Thread properties:

- Number of users: 800
- Ramp-up period (in sec): 2
- loop count: 1

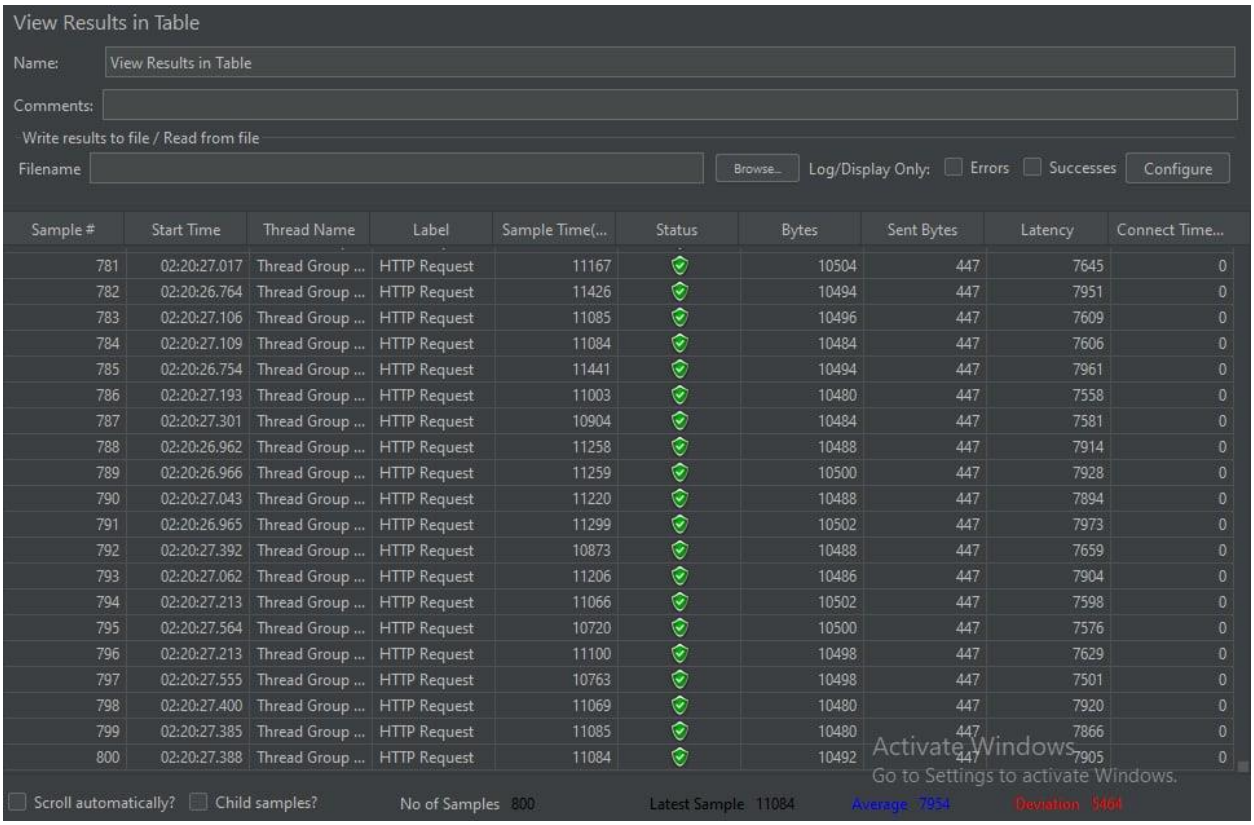


Figure: view result table for 800 users

Challenges & Solution

The IceHrm project faced several challenges during its implementation, most notably the difficulty of selecting appropriate scenarios for manual and automated testing, and problems interacting with elements within the IceHrm interface when using Catalon. Some unexpected errors also occurred during operation. To overcome these challenges, we carefully analyzed the system to select the most important scenarios and used element inspection tools to accurately identify elements.

Conclusion

The report documents the process of testing the quality and performance improvement of the open-source iceHrm system. The system manages human resources and organizes employee administrative processes. In this project, we conducted appropriate tests, including requirements analysis, automation, and performance testing, to ensure the system's performance.